

Dédicace

*Avant toute chose, je rends grâce à **Allah**, le Tout-Puissant, pour m'avoir accordé la force, la patience et la persévérance nécessaires pour surmonter les épreuves et mener ce travail à son accomplissement.*

Je dédie ce travail à mes chers parents, pour leur amour inconditionnel, leur soutien indéfectible et les sacrifices qu'ils ont consentis tout au long de mon parcours.

*Je le dédie également à toutes les personnes qui m'ont accompagné de près ou de loin durant cette étape importante de ma vie, ainsi qu'à **Oussama Belarch** et **Adil Agoussal** pour leur aide précieuse et leur immense soutien lors de la réalisation de ce projet.*

*Ce projet occupe une place particulière dans mon cœur. Bien plus qu'un simple projet d'ingénierie, il a été réalisé durant une période marquante de mon existence, celle où j'ai appris mon diagnostic de **cancer**. Dans les premiers jours de cette épreuve, alors que l'incertitude et les inquiétudes occupaient mon esprit, ce télescope est devenu bien plus qu'un ensemble de pièces mécaniques, électroniques et logicielles : il est devenu un refuge, une source de motivation et un moyen de continuer à avancer malgré les difficultés.*

Chaque étape de sa conception, chaque calcul, chaque pièce assemblée et chaque ligne de code écrite m'ont permis de garder le regard tourné vers l'avenir. Ce projet m'a offert des moments d'espoir lorsque j'en avais le plus besoin, et il gardera toujours une valeur unique, même parmi des réalisations futures plus ambitieuses.

*Je souhaite également exprimer ma profonde gratitude à mon oncologue, **Dr Zouhour Idder Fadoukhair**, pour son professionnalisme, sa bienveillance et le soutien qu'elle m'a apportés tout au long de mon parcours médical. Son accompagnement m'a donné la force et la sérénité nécessaires pour poursuivre mes études et mener ce projet à son terme.*

À toutes les personnes qui ont cru en moi, soutenu mes efforts et partagé cette aventure, je dédie humblement ce travail.

Remerciements

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué au succès de ce stage et de ce projet de fin d'études.

Mes remerciements s'adressent tout particulièrement à mon encadrant, **Monsieur Mohammed Bsiss**, pour son encadrement précieux, ses conseils judicieux et son accompagnement tout au long de la réalisation de ce projet de télescope autoguidé.

Je remercie également chaleureusement les équipes d'**Orange Digital Center Agadir** pour m'avoir ouvert les portes de leur FabLab, un espace propice à la créativité et à l'ingénierie, ainsi que **Moussasoft** et **Monsieur Moussa Lhoussin** pour leur expertise technique et la mise à disposition des composants électroniques indispensables à la concrétisation de ce projet.

Enfin, je remercie tout le corps professoral de la **Faculté des Sciences et Techniques de Tanger (FSTT)** pour la qualité de l'enseignement dispensé durant mes années d'études au sein du département de Génie Électrique.

Résumé

Ce rapport détaille la conception, la modélisation mathématique et la réalisation matérielle et logicielle d'un télescope Dobsonien autoguidé. L'objectif principal de ce projet est de rendre l'observation astronomique accessible en automatisant le pointage et le suivi des objets célestes. La partie mécanique a été conçue sur Onshape et imprimée en 3D, combinée à des éléments en plexiglass. Le système électronique s'articule autour de deux microcontrôleurs Arduino Nano séparant la commande des moteurs pas-à-pas de l'interface utilisateur afin d'éviter les parasites électromagnétiques. Plusieurs modes de fonctionnement ont été développés : un mode hors-ligne implémentant les équations de trigonométrie sphérique pour convertir les coordonnées équatoriales en coordonnées horizontales, un mode en ligne interfacé avec Stellarium via un script Python (Selenium), un mode manuel par joystick, et un mode de suivi compensant la rotation terrestre. Ce projet a été réalisé en partenariat avec Orange Digital Center et Moussasoft.

Abstract

This report details the design, mathematical modeling, and hardware/software implementation of an autoguided Dobsonian telescope. The main objective of this project is to make astronomical observation more accessible by automating the aiming and tracking of celestial objects. The mechanical structure was designed using Onshape, utilizing 3D printing and plexiglass components. The electronic system is based on two Arduino Nano microcontrollers, completely isolating the stepper motor drivers from the user interface and calculation unit to prevent electromagnetic interference. Several operational modes were developed : an offline mode implementing spherical trigonometry equations to convert equatorial coordinates to horizontal coordinates, an online mode interfaced with Stellarium via a Python scraping script (Selenium), a manual mode using a joystick, and a tracking mode to compensate for Earth's rotation. This project was carried out in partnership with Orange Digital Center and Moussasoft.

Liste des abréviations

Alt	Altitude (ou Hauteur)
Az	Azimut
API	Application Programming Interface
CAD	Computer-Aided Design (CAO : Conception Assistée par Ordinateur)
Dec	Déclinaison
GST	Greenwich Sidereal Time (Temps Sidéral de Greenwich)
GUI	Graphical User Interface (Interface Graphique Utilisateur)
HA	Hour Angle (Angle Horaire)
IDE	Integrated Development Environment (Environnement de Développement Intégré)
LST	Local Sidereal Time (Temps Sidéral Local)
ODC	Orange Digital Center
PWM	Pulse Width Modulation (Modulation par Largeur d'Impulsion)
RA	Right Ascension (Ascension Droite)
UT	Universal Time (Temps Universel)

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iii
Liste des abréviations	iv
Liste des figures	ix
Liste des tableaux	x
Introduction générale	1
1 Les Télescopes et Systèmes Autoguidés	3
1.1 Introduction	3
1.2 Les Télescopes	3
1.2.1 Histoire et principe optique général	3
1.2.2 Grandeurs caractéristiques	4
1.3 Types de télescopes	6
1.3.1 Télescope Réfracteur (Lunette astronomique)	6
1.3.2 Télescope Réflecteur (Newton)	8
1.4 Les Montures	9
1.4.1 Monture Équatoriale	9
1.4.2 Monture Dobsonienne (Alt-Az)	10
1.5 Les Télescopes Autoguidés	12
1.5.1 Objectifs du GoTo et de l'autoguidage	12
1.5.2 Principe de fonctionnement et architecture	12
1.6 Conclusion	13

2	Cadre du Stage : Les Entreprises d'Accueil	14
2.1	Introduction	14
2.2	Orange Digital Center Maroc	14
2.2.1	Présentation générale	14
2.2.2	Les quatre piliers d'ODC	15
2.2.3	ODC Agadir et apport au projet	15
2.3	Moussasoft	16
2.3.1	Histoire et fondateur	16
2.3.2	Catalogue et services	16
2.3.3	Apport au projet du télescope	16
2.4	Synergie ODC et Moussasoft	17
2.5	Organisation du stage	17
3	Conception et Réalisation	18
3.1	Introduction	18
3.2	Outils et Environnement de travail	18
3.2.1	Hardware	18
3.2.2	Software	20
3.3	Le Corps du Télescope (Body)	20
3.3.1	L'Optique et ses Mathématiques	20
3.3.2	La Base Azimutale (Z-Axis)	23
3.3.3	L'Axe d'Altitude (X-Axis)	26
3.3.4	Le Corps Télescopique : tubes et pièces 3D	28
3.3.5	Assemblage final du télescope	30
3.4	Les circuits :	31
3.4.1	Circuit 1 : Contrôle des moteurs	31
3.4.2	Circuit 2 : la télécommande	32
3.5	Programmation et Modes de Fonctionnement	35
3.5.1	Organigramme général du système	35
3.5.2	Le Mode Hors-Ligne (Offline) : Astronomie Sphérique	36
3.5.3	Le Mode En Ligne (Online) : Intégration Python & Stellarium	42
3.5.4	Le Mode de Suivi (Following Mode) - Ajout Algorithmique	45
3.5.5	Le Mode Manuel (Joystick)	49
3.6	Système d'alimentation et dimensionnement	51
3.6.1	Bilan de puissance détaillé	51
3.6.2	Solution principale : alimentation secteur 230 V / 12 V	51
3.6.3	Amélioration : alimentation sur batterie Li-ion pour usage nomade	52
3.6.4	Architecture de l'alimentation	53
3.7	La Touche Finale et Résultats	54

3.7.1	Le support laser vert (« Finder »)	54
3.7.2	LOGO	54
3.7.3	Tests sur le terrain et observations réalisées	55
3.7.4	Précision de pointage mesurée	55
3.7.5	Photos du prototype et des observations	57
3.7.6	Perspectives d'amélioration	58
Conclusion Générale		59
Bibliographie		60
A Code complet : Arduino n°1 (Télécommande)		61
Annexe A : Code Arduino n°1		61
B Code complet : Arduino n°2 (Pilote des moteurs)		67
Annexe B : Code Arduino n°2		67
C Code complet : Application Python (Stellarium Scraper)		70
Annexe C : Code Python		70
D Schémas de câblage détaillés		74
Annexe D : Schémas de câblage		74
D.1	Brochage Arduino n°1	74
D.2	Brochage Arduino n°2	75
D.3	Câblage A4988	75
D.4	Connecteur télécommande ↔ télescope	75

Table des figures

1.1	Galileo Galilei Telescope by hulton archiva	3
1.2	Les aberrations chromatiques dans un telescope à lentilles et telescope à miroir	4
1.3	distance focal	5
1.4	Schéma de principe d'un télescope réfracteur (Lunette).	7
1.5	Exemple commercial d'un télescope réfracteur achromatique.	7
1.6	Schéma de principe d'un télescope de Newton	8
1.7	Télescope de Newton commercial sur monture Équatoriale	9
1.8	Principe de la monture équatoriale	10
1.9	une monture équatoriale allemande	10
1.10	Principe cinématique d'une monture Alt-Azimutale.	11
1.11	Architecture fonctionnelle générique d'un système d'autoguidage.	12
2.1	Orange Digital Center logo	14
2.2	Vue du FabLab d'Orange Digital Center Agadir.	15
2.3	Moussasoft logo	16
3.1	Schéma des mathématiques optiques utilisées pour le dimensionnement . . .	21
3.2	Marche des rayons dans le télescope de Kepler : un faisceau parallèle (étoile à l'infini) entre sous un angle α , converge au foyer commun $F'_1 = F_2$, et ressort parallèle sous un angle $\alpha' = G \cdot \alpha$	22
3.3	Résultat du zoom obtenu	23
3.4	Vue isométrique de la base azimutale modélisée sur Onshape.	24
3.5	Vue éclatée de la base azimutale révélant l'agencement des roulements, du pignon moteur et de la courroie crantée.	24
3.6	la base azimutale assemblée.	25
3.7	Modélisation CAO de l'axe d'altitude sur Onshape.	26
3.8	demi-couronne de 96 dents et pignon de 11 dents	27
3.9	Pliage à chaud des plaques en plexiglass formant les supports latéraux. . .	28
3.10	la base azimutale et altitude assemblée.	28

3.11	Vue CAO des tubes télescopiques coulissants et de leurs pièces de raccordement imprimées en 3D.	29
3.12	Porte-objectif et oculaire	29
3.13	vue interne du corps du télescope	30
3.14	Vue d'ensemble du télescope assemblé.	30
3.15	Schéma synoptique simplifié du circuit n°1 (contrôle des moteurs)	31
3.16	isolateur numérique (ISO7221)	33
3.17	Schéma synoptique simplifié du circuit n°2 (Télécommande)	34
3.18	Modélisation CAO du boîtier de télécommande.	35
3.19	Boîtier de télécommande finalisé, imprimé en PLA.	35
3.20	Organigramme global du firmware de télescope.	36
3.21	Sphère céleste et triangle de position sphérique PZS liant le pôle céleste P , le zénith Z et l'étoile S . Le côté $PZ = 90^\circ - \phi$ (colatitude), $PS = 90^\circ - Dec$ (distance polaire), $ZS = 90^\circ - Alt$ (distance zénithale). L'angle au sommet P est l'angle horaire HA	37
3.22	Organigramme du mode hors-ligne.	40
3.23	Représentation polaire locale (Azimut, Altitude radiale vers le centre) de la cible Capella. $Alt \approx 73.97^\circ$, $Az \approx 347.99^\circ$, observée depuis Agadir (Lat = $30.4^\circ N$, Long = -9.6°) pour $d = 10995$	41
3.24	Diagramme de séquence du mode en ligne (Online).	42
3.25	Capture d'écran de l'interface Python (Tkinter).	43
3.26	Trame série envoyée à l'Arduino pour l'étoile Capella (Azimut : $34759'24''$, Altitude : $7358'12''$).	44
3.27	Organigramme du mode de suivi (Following Mode)	48
3.28	Trajectoire complète de Capella depuis Agadir (latitude $30,4^\circ N$).	49
3.29	La validité de l'algorithme sur Stellarium.	49
3.30	Architecture simplifié du circuit d'alimentation : $12\text{ V} \rightarrow$ drivers (direct) et $\rightarrow 5\text{ V}$ pour la logique (arduino).	53
3.31	Laser de guidage	54
3.32	LOGO	55
3.33	Erreur de pointage en mode en ligne (bleu) et hors-ligne (rouge) pour 10 cibles célestes depuis Agadir.	56
3.34	Télescope assemblé au FabLab et en cours de test nocturne à Agadir.	57
3.35	À gauche : photographie de la Lune réalisée à travers l'oculaire avec un smartphone. Au centre : cliché de la planète Saturne. À droite : comète C/2023 A3 capturée pendant son passage en octobre 2024.	57

Liste des tableaux

1.1	Grandeurs optiques calculées pour notre télescope (objectif $f_{obj} = 1000$ mm, $D = 50$ mm).	6
1.2	Avantages et inconvénients des télescopes réfracteurs.	7
1.3	Comparatif des configurations de télescopes réflecteurs.	9
1.4	Comparaison des montures équatoriale et Dobsonienne.	11
2.1	Planning prévisionnel du stage.	17
3.1	Nomenclature matérielle complète (BOM).	19
3.2	Chaîne logicielle utilisée.	20
3.3	Bilan optique théorique du télescope.	23
3.4	Affectation des broches de l'Arduino Nano n°2 (contrôle moteurs).	32
3.5	Catalogue des 20 étoiles embarquées dans le firmware.	39
3.6	Performances du mode manuel.	51
3.7	Bilan détaillé des consommations électriques.	51
3.8	Comparaison des solutions de batterie pour usage nomade.	53
D.1	Brochage complet de l'Arduino n°1 (Télécommande).	74
D.2	Brochage complet de l'Arduino n°2 (Pilote moteurs).	75

Introduction générale

L'astronomie est l'une des sciences les plus anciennes, guidant l'humanité depuis des millénaires. Au Maroc, l'observation du ciel nocturne bénéficie de conditions géographiques et climatiques très favorables, notamment dans les régions du sud et de l'Atlas. Toutefois, la pratique de l'astronomie amateur se heurte souvent à une barrière technique majeure : le repérage et le suivi des objets célestes. Face à l'immensité de la voûte céleste, l'alignement manuel d'un télescope requiert de l'expérience, de la patience, et une solide connaissance du ciel.

Problématique : L'observation prolongée ou l'astrophotographie (qui nécessite l'accumulation ou "stacking" de lumière) exige de compenser la rotation terrestre avec une précision redoutable. Les montures équatoriales motorisées commerciales qui remplissent ce rôle sont souvent onéreuses, complexes à calibrer pour un novice, et encombrantes. Les montures alt-azimutales (de type Dobsonien), bien que mécaniquement simples et robustes, ne permettent pas un suivi intuitif car les deux axes doivent bouger à des vitesses variables simultanément.

Objectifs du projet : Ce projet de stage a pour ambition de démocratiser l'accès à l'astronomie en proposant l'étude et la réalisation d'un **télescope Dobsonien auto-guidé** (système GoTo open-source). L'objectif est de concevoir un dispositif mécatronique abordable combinant l'impression 3D, la programmation embarquée (Arduino), la trigonométrie sphérique et l'exploitation de données web (Stellarium) pour automatiser le pointage et le suivi des astres.

Méthodologie : Le développement du projet s'est articulé autour d'une approche systémique. La conception mécanique (CAO) a été réalisée sur Onshape. L'optique a été dimensionnée sur la base d'équations de conjugaison standard. L'électronique s'est construite sur une architecture maître/esclave avec deux cartes Arduino Nano pour scinder le calcul pur (interface et trigonométrie) de la commande de puissance (moteurs pas-à-pas NEMA 17). Enfin, la programmation a fait appel à l'environnement Arduino (C++) pour l'embarqué et Python pour la passerelle logicielle.

Plan du rapport : Ce rapport s'organise en trois grands chapitres. Le premier chapitre dresse un état de l'art sur les télescopes, leurs montures et les principes d'autoguidage. Le deuxième chapitre présente les entreprises d'accueil, Orange Digital Center et Moussasoft, ayant soutenu ce projet. Le troisième chapitre, cœur de ce mémoire, détaille

minutieusement la conception optique, mécanique, électrique et logicielle du télescope, incluant l'analyse mathématique des modes de fonctionnement (hors-ligne, en ligne, suivi et manuel).

Chapitre 1

Les Télescopes et Systèmes Autoguidés

1.1 Introduction

Ce chapitre a pour vocation de poser les bases théoriques de l'optique instrumentale et de l'astronomie d'observation. Nous y explorerons les différents types de télescopes, les montures qui les supportent, ainsi que les principes fondamentaux qui régissent les systèmes autoguidés.

1.2 Les Télescopes

1.2.1 Histoire et principe optique général

Le télescope a révolutionné notre compréhension de l'univers depuis les premières observations de Galileo Galilei en 1609, lorsque celui-ci pointa pour la première fois une lunette artisanale vers la voûte céleste et y découvrit les quatre principales lunes de Jupiter (Io, Europe, Ganymède et Callisto).



FIGURE 1.1 – Galileo Galilei Telescope by hulton archiva

Cette découverte fonda la révolution copernicienne et démontra que tous les corps célestes ne tournent pas autour de la Terre. Quelques décennies plus tard, Isaac Newton, frustré par les aberrations chromatiques inhérentes aux lunettes à lentilles, conçut en 1668 le premier télescope à miroir, ouvrant la voie aux grands instruments modernes [?].

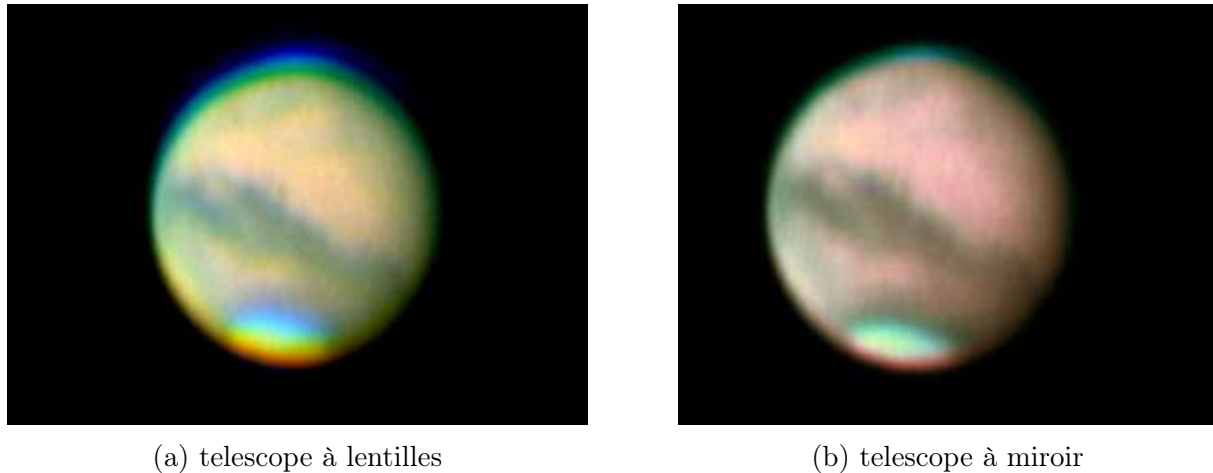


FIGURE 1.2 – Les aberrations chromatiques dans un télescope à lentilles et télescope à miroir

Son principe repose sur la collecte de la lumière (via un objectif réfractif ou un miroir primaire réfléchissant) pour former une image dans le plan focal, qui est ensuite agrandie par un oculaire fonctionnant comme une loupe. L'objectif d'un instrument d'astronomie n'est pas seulement de grossir, mais surtout d'augmenter la quantité de lumière perçue (le pouvoir collecteur, proportionnel à D^2) et de séparer les détails angulaires les plus fins (le pouvoir séparateur).

Définition : Plan focal

On appelle plan focal le lieu géométrique des points où convergent les rayons issus d'un objet situé à l'infini après leur passage à travers le système optique. Pour un astre situé à des années-lumière de la Terre, les rayons incidents arrivent quasi parallèles, et l'image se forme exactement à la distance focale f du système.

1.2.2 Grandeurs caractéristiques

Tout instrument optique astronomique se définit par plusieurs paramètres mathématiques fondamentaux dont l'analyse rigoureuse conditionne le choix des composants et la qualité de l'image finale. Nous présentons ci-dessous les principales grandeurs, avec leurs formulations analytiques.

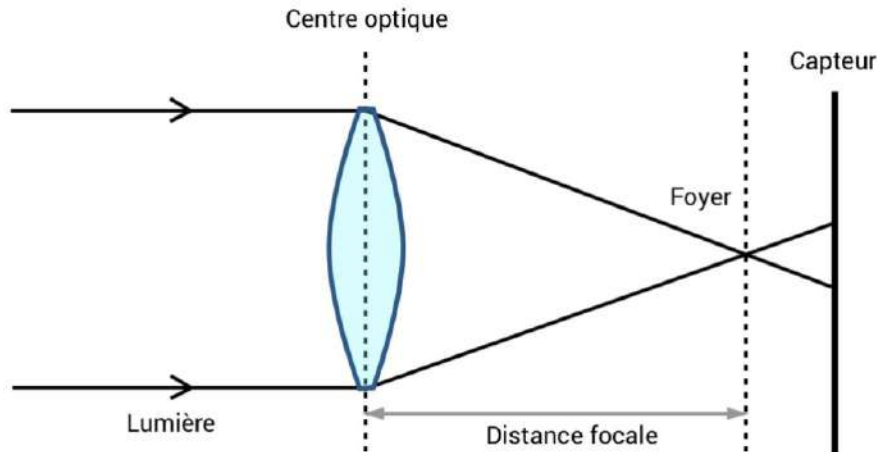


FIGURE 1.3 – distance focal

- **La distance focale (f)** : Distance entre la lentille (ou le miroir primaire) et le plan focal où se forme l'image d'un objet situé à l'infini. Elle s'exprime en millimètres et conditionne directement le grossissement.
- **L'ouverture (D)** : Le diamètre de l'objectif (ou du miroir primaire), exprimé en mm. C'est le paramètre le plus déterminant car il fixe à la fois le pouvoir collecteur et le pouvoir séparateur.
- **Le rapport f/D (ou nombre d'ouverture)** : Détermine la "rapidité" optique. Un instrument court ($f/D = 4$ à 6) est dit *rapide* et favorise l'astrophotographie du ciel profond ; un instrument long ($f/D = 10$ à 15) favorise les hauts grossissements planétaires.
- **Le grossissement (G)** :

$$G = \frac{f_{\text{objectif}}}{f_{\text{oculaire}}} \quad (1.1)$$

Notons qu'au-delà d'un *grossissement utile maximal* approximé par $G_{\text{max}} \approx 2 \cdot D$ (avec D en mm), l'image se dégrade par dilution lumineuse sans gain de détail.

- **Le pouvoir séparateur** (Formule de Dawes), exprimé en secondes d'arc :

$$P_s = \frac{116}{D \text{ (en mm)}} \quad (1.2)$$

Cette limite empirique caractérise la séparation minimale entre deux étoiles doubles que l'instrument peut résoudre. Elle découle du critère de Rayleigh appliqué à la diffraction par une ouverture circulaire :

$$\theta_{\text{Rayleigh}} = 1,22 \frac{\lambda}{D} \quad (1.3)$$

où λ est la longueur d'onde de la lumière (typiquement 550 nm pour le visible).

— **Le pouvoir collecteur** (relatif à l'œil humain dont la pupille atteint ~ 7 mm) :

$$P_c = \left(\frac{D_{\text{télescope}}}{D_{\text{œil}}} \right)^2 = \left(\frac{D}{7} \right)^2 \quad (1.4)$$

— **La magnitude limite** accessible avec l'instrument :

$$m_{\text{lim}} = 2,1 + 5 \log_{10}(D_{\text{mm}}) \quad (1.5)$$

TABLE 1.1 – Grandeurs optiques calculées pour notre télescope (objectif $f_{\text{obj}} = 1000$ mm, $D = 50$ mm).

Grandeur	Formule	Valeur
Grossissement G	$f_{\text{obj}}/f_{\text{oc}}$	$\approx 12\times$
Rapport f/D	$1000/50$	$f/20$
Pouvoir séparateur de Dawes	$116/D$	$\approx 2,32''$
Critère de Rayleigh	$1,22\lambda/D$	$\approx 2,77''$
Pouvoir collecteur	$(D/7)^2$	$\approx 51\times$
Magnitude limite	$2,1 + 5 \log_{10}(D)$	$\approx 10,6$

1.3 Types de télescopes

1.3.1 Télescope Réfracteur (Lunette astronomique)

La lunette astronomique, ou télescope réfracteur, utilise une combinaison de lentilles convergentes pour réfracter et focaliser la lumière incidente. C'est le plus ancien type d'instrument optique astronomique, inventé indépendamment par plusieurs opticiens hollandais au début du XVII^e siècle, puis perfectionné par Galilée et Kepler. Sa configuration la plus courante (configuration képlérienne) repose sur deux lentilles convergentes : un objectif de grande focale et un oculaire de courte focale, séparées d'une distance égale à $f_{\text{obj}} + f_{\text{oc}}$.

L'image formée au foyer de l'objectif est une image réelle, renversée, vue ensuite à travers l'oculaire qui agit comme une loupe. Le grossissement angulaire est alors donné par $G = f_{\text{obj}}/f_{\text{oc}}$.

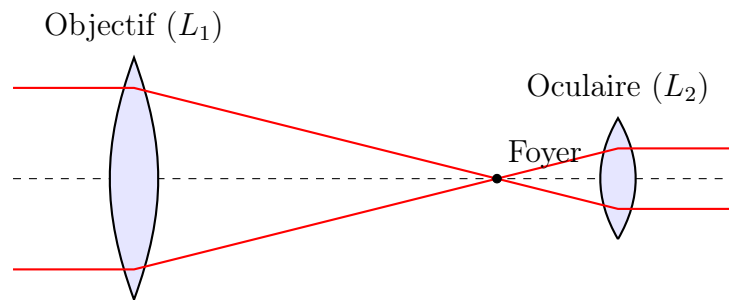


FIGURE 1.4 – Schéma de principe d'un télescope réfracteur (Lunette).

Les réfracteurs offrent un bon contraste, une excellente stabilité (tube fermé limitant les courants thermiques) et une absence d'obstruction centrale. Toutefois, ils souffrent d'**aberrations chromatiques** (halo coloré dû à la dispersion : les longueurs d'onde courtes — bleu — sont plus déviées que les rouges) à moins d'utiliser des doublets achromatiques (verres crown et flint accolés) ou des triplets apochromatiques très onéreux. De plus, leur diamètre maximal est limité car au-delà de ~ 200 mm, le poids du verre déforme l'objectif sous son propre poids.

TABLE 1.2 – Avantages et inconvénients des télescopes réfracteurs.

Avantages	Inconvénients
Excellent contraste et netteté	Aberration chromatique
Pas d'obstruction centrale	Coût élevé pour grands diamètres
Tube fermé (peu de turbulence interne)	Tube long et encombrant
Robuste et stable mécaniquement	Poids important
Peu d'entretien (pas de réalignement)	Diamètres limités à ~ 200 mm



FIGURE 1.5 – Exemple commercial d'un télescope réfracteur achromatique.

1.3.2 Télescope Réflecteur (Newton)

Inventé par Isaac Newton, ce télescope utilise un miroir primaire parabolique ou sphérique pour concentrer la lumière, et un miroir secondaire plan pour dévier le faisceau vers l'oculaire.

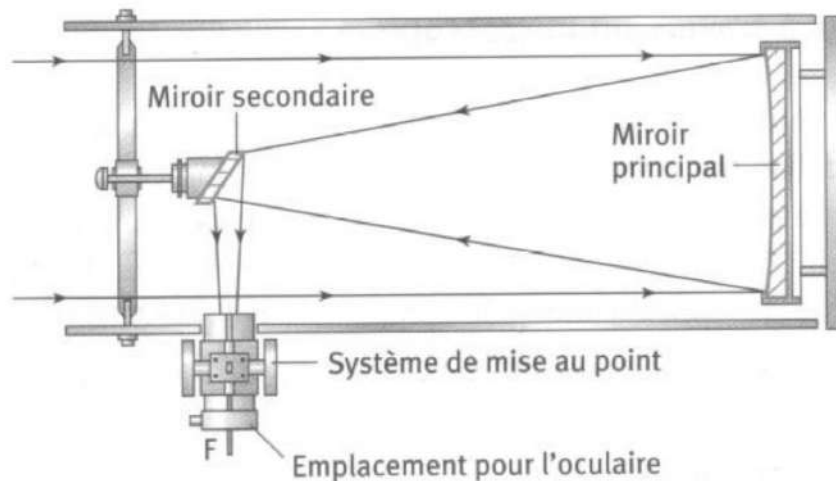


FIGURE 1.6 – Schéma de principe d'un télescope de Newton

Il permet de grands diamètres à moindre coût et ne présente pas d'aberration chromatique car la réflexion sur un miroir est indépendante de la longueur d'onde. En revanche, il introduit des aberrations propres : **coma** (élongation des étoiles hors-axe), **astigmatisme**, et **aberration de sphéricité** si le miroir n'est pas parfaitement parabolique. La présence d'un miroir secondaire crée également une **obstruction centrale** qui réduit le contraste et provoque des aigrettes de diffraction caractéristiques autour des étoiles brillantes.

Plusieurs variantes existent :

- **Newton** : miroir primaire parabolique + secondaire plan à 45° (configuration choisie pour notre projet).
- **Cassegrain** : miroir primaire parabolique troué + secondaire hyperbolique convexe renvoyant la lumière à travers le primaire.
- **Schmidt-Cassegrain** : ajoute une lame correctrice à l'entrée pour corriger l'aberration sphérique.
- **Ritchey-Chrétien** : variante du Cassegrain avec primaire et secondaire hyperboliques, élimine le coma — adoptée par le Hubble Space Telescope.

TABLE 1.3 – Comparatif des configurations de télescopes réflecteurs.

Type	Primaire	Secondaire	Compact	Coût
Newton	Parabolique	Plan	Non	Faible
Cassegrain	Parabolique	Hyperbolique	Oui	Moyen
Schmidt-Cassegrain	Sphérique + lame	Sphérique	Très	Élevé
Ritchey-Chrétien	Hyperbolique	Hyperbolique	Oui	Très élevé



FIGURE 1.7 – Télescope de Newton commercial sur monture Équatoriale

1.4 Les Montures

1.4.1 Monture Équatoriale

La monture équatoriale possède un axe aligné parallèlement à l'axe de rotation de la Terre, c'est-à-dire pointant vers le pôle céleste nord (matérialisé en pratique par l'étoile Polaris, α Ursae Minoris). Cet alignement, appelé *mise en station polaire*, est l'étape la plus délicate de l'usage d'une telle monture : toute erreur d'azimut ou de hauteur du pôle est répercutée comme dérive systématique des coordonnées.

Pour compenser la rotation terrestre, il suffit de motoriser un seul axe (Ascension Droite, ou RA) à une vitesse angulaire constante :

$$\omega_{\text{sidérale}} = \frac{360^\circ}{23 \text{ h } 56 \text{ min } 4,1 \text{ s}} \approx 15,04107^\circ/\text{heure} \approx 4,178 \times 10^{-3} \text{ }^\circ/\text{s} \quad (1.6)$$

On distingue deux grandes familles : la **monture allemande** (axes en croix avec contre-poids) et la **monture à fourche** (le tube est encastré entre deux bras pivotant autour de l'axe polaire).

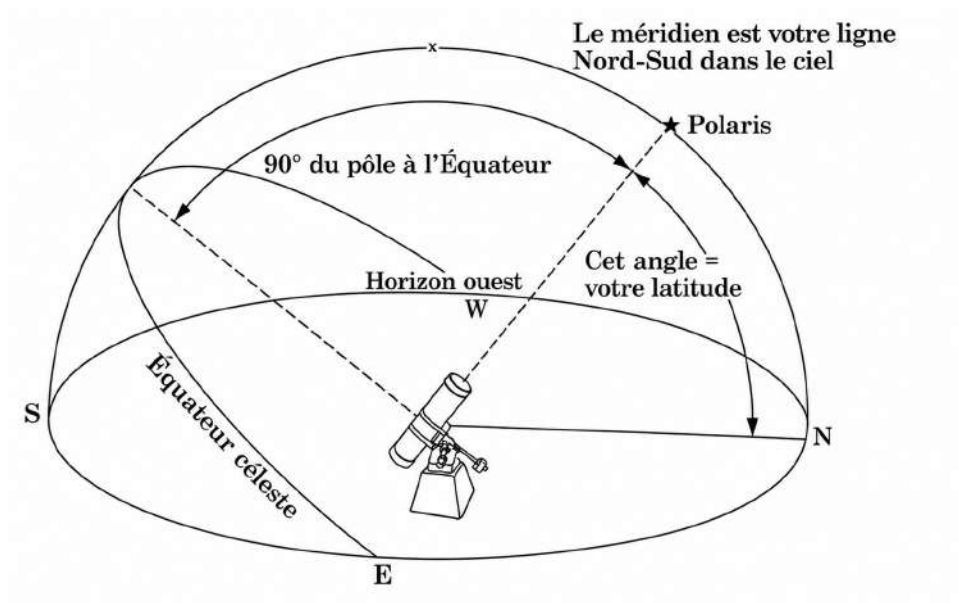


FIGURE 1.8 – Principe de la monture équatoriale



FIGURE 1.9 – une monture équatoriale allemande

1.4.2 Monture Dobsonienne (Alt-Az)

La monture de type Dobson (ou Alt-Azimutale) repose sur deux axes perpendiculaires : l'azimut (rotation horizontale) et l'altitude (mouvement vertical).

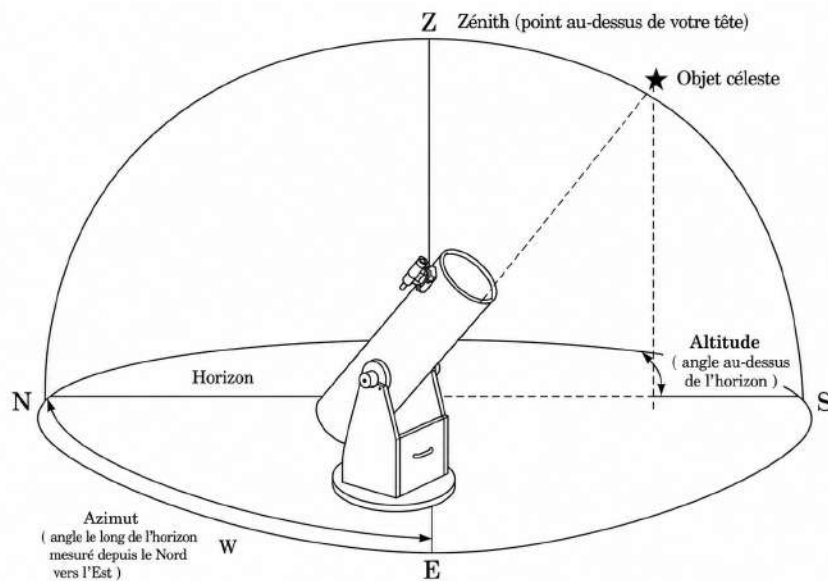


FIGURE 1.10 – Principe cinématique d’une monture Alt-Azimutale.

Elle est **extrêmement stable et facile à fabriquer**, ce qui en a fait la solution privilégiée des astronomes amateurs depuis sa popularisation par John Dobson dans les années 1970. Son architecture s’apparente à celle d’un canon : un axe vertical d’azimut (rotation autour de la verticale du lieu) et un axe horizontal d’altitude (tangage). Toutefois, pour suivre une étoile à travers le ciel, il faut faire varier *simultanément* l’altitude et l’azimut à des vitesses *non constantes* (et même non monotones près du méridien), requérant un calculateur embarqué — c’est précisément l’objet de notre projet d’autoguidage.

TABLE 1.4 – Comparaison des montures équatoriale et Dobsonienne.

Critère	Équatoriale	Dobsonienne (Alt-Az)
Coût	Élevé	Faible
Complexité mécanique	Élevée (contrepoids, axes obliques)	Très faible (deux paliers)
Suivi sidéral	1 moteur à vitesse constante	2 moteurs à vitesse variable
Mise en station	Indispensable (vers le pôle)	Aucune
Champ de rotation	Aucun	Présent (rotation d’image)
Adapté astrophoto	Oui	Difficile (sans dérotateur)
longue pose		
Adapté observation visuelle	Oui	Excellent
Stabilité	Moyenne (porte-à-faux)	Excellente (centre de gravité bas)

1.5 Les Télescopes Autoguidés

1.5.1 Objectifs du GoTo et de l'autoguidage

L'autoguidage permet de pointer automatiquement des objets peu lumineux, invisibles à l'œil nu. En astrophotographie, le suivi permet le "stacking" (empilement de poses courtes). Le rapport signal sur bruit (SNR) s'améliore selon la formule :

$$SNR \propto \sqrt{N} \quad (1.7)$$

où N est le nombre de poses. Sans suivi, les étoiles laisseraient des traînées lumineuses (star trails).

1.5.2 Principe de fonctionnement et architecture

Le système nécessite trois informations sources : (i) les coordonnées équatoriales de la cible (Ascension droite RA et Déclinaison Dec), (ii) une horloge précise permettant de calculer le Temps Sidéral local, et (iii) la géolocalisation (latitude ϕ , longitude λ) de l'observateur. Ces données alimentent une transformation trigonométrique sphérique (matrice de passage du repère équatorial au repère horizontal local), dont la sortie commande des drivers de moteurs pas-à-pas via un microcontrôleur embarqué.

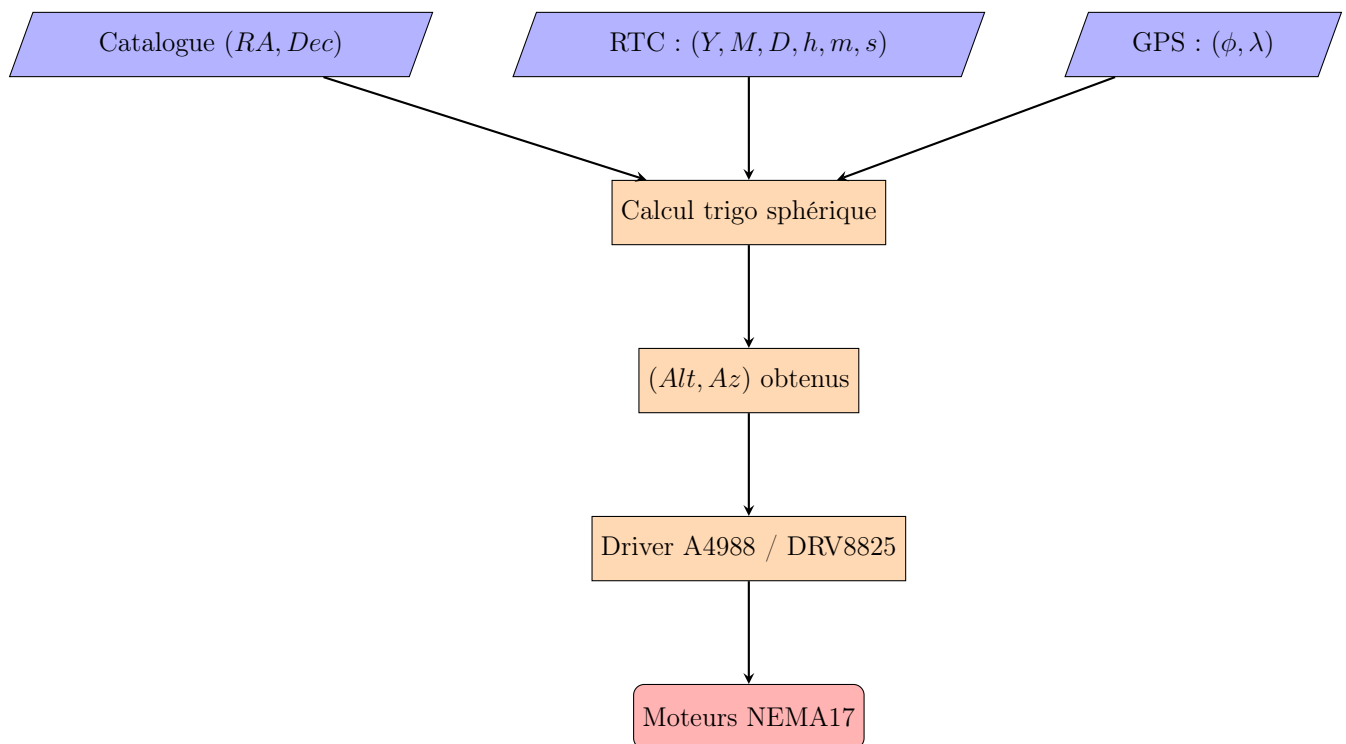


FIGURE 1.11 – Architecture fonctionnelle générique d'un système d'autoguidage.

L'autoguidage différentiel — par opposition au simple GoTo — implique une boucle

de correction permanente afin de compenser les dérives mécaniques (jeu d'engrenages, glissement de la courroie) et la rotation terrestre. Dans les systèmes les plus avancés, une caméra auxiliaire surveille une étoile-guide et envoie des micro-corrrections via un protocole ST4 ; dans notre prototype, ce rôle est joué par le calcul trigonométrique réactualisé toutes les secondes.

1.6 Conclusion

Ce chapitre a permis d'établir les bases théoriques de l'instrumentation astronomique : nous avons défini les grandeurs caractéristiques (focale, ouverture, grossissement, pouvoir séparateur), comparé les architectures réfractrices et réflectrices, et analysé les deux grandes familles de montures (équatoriale et alt-azimutale). Le choix d'une monture Dobsonienne pour notre projet — motivée par sa simplicité mécanique et son faible coût — impose en contrepartie une complexité logicielle importante : le suivi nécessite une commande différentielle sur deux axes simultanément, ce qui sera l'objet du chapitre 3.

Chapitre 2

Cadre du Stage : Les Entreprises d'Accueil

2.1 Introduction

La réalisation d'un projet mécatronique d'envergure tel qu'un télescope autoguidé exige des ressources matérielles, logicielles et humaines qu'un étudiant ne possède que rarement à titre individuel. C'est dans ce contexte que mon stage s'est déroulé en partenariat avec deux entités complémentaires implantées au Maroc : **Orange Digital Center (ODC)** et **Moussasoft**. La première a fourni l'environnement de prototypage et la communauté technique, la seconde a fourni les composants critiques et l'expertise en intégration électronique.

2.2 Orange Digital Center Maroc

2.2.1 Présentation générale

Orange Digital Center (ODC) est un programme mondial déployé par le groupe Orange en partenariat avec ses filiales locales, visant à accompagner la transformation numérique et soutenir l'innovation inclusive. Le réseau international compte aujourd'hui plus d'une vingtaine de centres en Afrique, au Moyen-Orient et en Europe. Au Maroc, ODC a établi des sites à Rabat, Casablanca, Tanger, Fès et Agadir, où s'est déroulé mon stage.



FIGURE 2.1 – Orange Digital Center logo

2.2.2 Les quatre piliers d'ODC

1. **École du Code** : formations gratuites au développement web, mobile, à l'intelligence artificielle et à la cybersécurité.
2. **FabLab Solidaire** : atelier de fabrication numérique ouvert aux porteurs de projet, équipé d'imprimantes 3D (FDM et SLA), de découpes laser CO₂, de fraiseuses CNC et d'un matériel d'électronique professionnel.
3. **Orange Fab** : accélérateur de start-up offrant un accompagnement business et technologique.
4. **Orange Ventures** : fonds d'investissement en capital-risque pour les start-up matures.

2.2.3 ODC Agadir et apport au projet

Le site d'Agadir, situé au cœur du Souss-Massa, dispose d'un FabLab particulièrement bien équipé. Pour le projet de télescope autoguidé, les ressources suivantes ont été mises à disposition :

- Imprimantes 3D **Creality Ender-3 V2** et **Prusa i3 MK3S** pour la fabrication des engrenages, du support laser et des pièces de raccordement des tubes.
- Découpeuse laser pour la mise en forme des plateaux en plexiglass de la base azimutale et de l'axe d'altitude.
- Postes de soudure et instruments de mesure (oscilloscopes Rigol, multimètres Fluke, alimentations de laboratoire).
- Espace de travail collaboratif et accompagnement par les coachs techniques d'ODC.

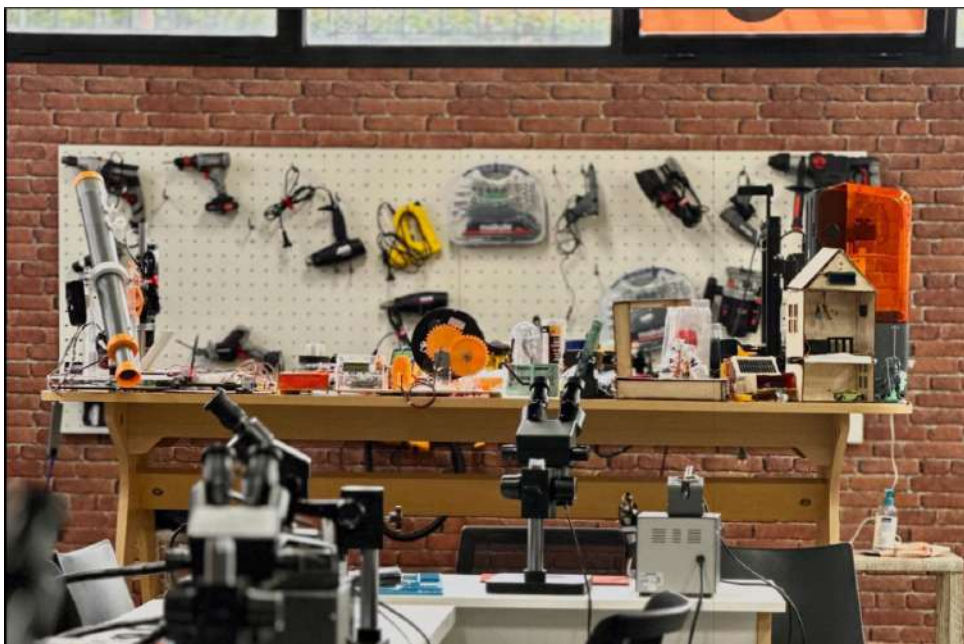


FIGURE 2.2 – Vue du FabLab d'Orange Digital Center Agadir.

2.3 Moussasoft

2.3.1 Histoire et fondateur

Moussasoft (<https://www.moussasoft.com/>) est une entreprise marocaine fondée par l'ingénieur-entrepreneur **Moussa Lhoussin**. À l'origine engagée dans le développement de solutions logicielles et matérielles sur-mesure, Moussasoft s'est progressivement positionnée comme un acteur de référence dans la distribution de composants électroniques et de kits éducatifs en robotique au Maroc, tout en continuant à concevoir et fabriquer des projets complexes pour des clients industriels et académiques.



FIGURE 2.3 – Moussasoft logo

2.3.2 Catalogue et services

Le catalogue de Moussasoft comprend des composants essentiels et de qualité pour les projets de mécatronique :

- Cartes microcontrôleurs : **Arduino Uno, Nano, Mega 2560, Due, Teensy 4.1, ESP32, Raspberry Pi.**
- Drivers moteurs : **A4988, DRV8825, L298N, TB6600.**
- Moteurs : pas-à-pas (NEMA 8, 14, 17, 23), servomoteurs, moteurs DC.
- Capteurs : IMU (MPU-6050, MPU-9250), GPS, ultrasoniques, optiques.
- Affichages : LCD I2C, écrans OLED, écrans TFT.
- Kits robotiques éducatifs et imprimantes 3D.
- Conseil technique et support à la conception.

2.3.3 Apport au projet du télescope

Pour ce projet, Moussasoft a apporté :

1. La fourniture des deux cartes **Arduino Nano** et des drivers **A4988** configurés en microstepping 1/32.
2. Les moteurs pas-à-pas **NEMA 17** (1,8°/pas, couple nominal 0,4 N·m) qui actionnent les deux axes.

3. Une expertise précieuse sur l'**isolation des masses** et la séparation logique/puissance, recommandation cruciale qui a conduit à l'architecture à deux Arduino communiquant par lignes logiques au lieu d'un système monolithique sensible aux parasites EM.
4. Le LCD I2C 16×2 et le joystick analogique de la télécommande.

2.4 Synergie ODC et Moussasoft

La synergie entre les capacités de prototypage du FabLab d'Orange Digital Center et l'expertise en intégration électronique de Moussasoft a permis de mener à bien ce projet d'ingénierie complexe, alliant mécanique, électronique et développement logiciel. La complémentarité de ces deux structures illustre un écosystème marocain naissant et dynamique dans lequel makers, étudiants, entrepreneurs et industriels collaborent autour de l'innovation matérielle (*hardware*). Sans cette synergie, la durée et le coût de réalisation de ce prototype auraient été significativement plus élevés, et la qualité du résultat final substantiellement inférieure.

2.5 Organisation du stage

Le stage s'est déroulé sur une période de plusieurs semaines, articulée selon les phases suivantes :

TABLE 2.1 – Planning prévisionnel du stage.

Phase	Activités	Durée (sem.)
1	État de l'art et étude bibliographique (optique, montures, codes existants)	2
2	Conception CAO sur Onshape, dimensionnement optique et mécanique	3
3	Prototypage : impression 3D, découpe laser, assemblage mécanique	3
4	Câblage électronique, intégration des deux Arduino et drivers	2
5	Développement logiciel (Arduino C++ et Python/Tkinter/Selenium)	3
6	Tests sur le terrain, calibrage, observation	2
7	Rédaction du rapport et préparation de la soutenance	2

Chapitre 3

Conception et Réalisation

3.1 Introduction

Après avoir présenté les fondements théoriques des télescopes et des systèmes d'autoguidage dans le premier chapitre, puis décrit le cadre de réalisation du projet et les structures d'accueil dans le deuxième chapitre, ce troisième chapitre est consacré à la conception et à la réalisation pratique du télescope Dobsonien autoguidé.

Cette partie constitue le cœur du projet puisqu'elle détaille l'ensemble des choix techniques ayant conduit à la réalisation du prototype. Elle couvre successivement l'étude optique ayant permis le dimensionnement du système d'observation, la conception mécanique de la monture et du corps du télescope, le développement de l'architecture électronique assurant la commande des moteurs ainsi que l'implémentation logicielle des différents modes de fonctionnement. Une attention particulière est portée aux calculs et aux modèles mathématiques utilisés pour le pointage et le suivi des objets célestes.

L'objectif de ce chapitre est de présenter de manière méthodique les différentes étapes de conception, de fabrication et d'intégration qui ont permis de transformer les concepts théoriques étudiés précédemment en un système fonctionnel capable d'automatiser l'observation astronomique.

3.2 Outils et Environnement de travail

3.2.1 Hardware

La nomenclature matérielle (BOM, Bill of Materials) constitue la première étape de tout projet d'ingénierie. Le tableau 3.1 en donne la liste exhaustive, ainsi que le rôle fonctionnel de chaque composant et son origine d'approvisionnement.

TABLE 3.1 – Nomenclature matérielle complète (BOM).

Composant	Qté	Rôle	Fournisseur
Arduino Nano (ATmega328P)	2	Contrôle moteurs + interface/calcul	Moussasoft
Driver A4988	2	Interface puissance steppers (1/32 μ s)	Moussasoft
Moteur NEMA 17 (1,7 A, 0,4 N·m)	2	Actionneurs Az / Alt	Moussasoft
LCD I2C 16×2 (PCF8574)	1	Affichage de l'IHM	Moussasoft
Joystick analogique 2 axes + SW	1	Contrôle manuel / navigation UI	Moussasoft
Boutons-poussoirs	2	Mode + Réglage date/heure	Moussasoft
Fin de course (limit switch)	1	Homing Az	FabLab ODC
Alimentation 12 V / 3 A	1	Énergie moteurs et logique	FabLab ODC
Fusible 3 A	1	Protection court-circuit	FabLab ODC
Plexiglass 5 mm	$\approx 1 \text{ m}^2$	Plateaux Az et Alt	FabLab ODC
Filament PLA 1,75 mm	$\approx 1 \text{ kg}$	Engrenages, supports, boîtier	3dware
Courroie crantée GT2	1 m	Couronne dentée azimutale	Moussasoft
Roulements à billes 608ZZ	4	Liaisons pivot Az / Alt	stock personnel
Lentille convergente +1 D	1	Objectif ($f = 1 \text{ m}$)	Optique locale
Lentille convergente +12 D	1	Oculaire ($f = 83 \text{ mm}$)	Optique locale
Tubes PVC $\varnothing 50 / 63 \text{ mm}$	2	Corps optique télescopique	stock personnel
Laser vert 5 mW	1	Pointage manuel et calibrage	aliexpress
Câbles, vis M3/M4, écrous	-	Visserie et câblage	bricoma

Remarque

Le choix de **deux cartes Arduino Nano** (au lieu d'une seule plus puissante comme la Mega) n'est pas dicté par la limitation de RAM ou de Flash : il vise à *isoler galvaniquement* le calculateur (Arduino n°1, dans la télécommande) de l'étage de puissance (Arduino n°2, près des drivers et moteurs). Les commutations rapides des A4988 (chopper à $\sim 50 \text{ kHz}$ et 12V) génèrent des transitoires sur les rails 5 V et la masse, capables de perturber la lecture analogique du joystick et l'écran LCD via le bus I2C.

3.2.2 Software

Le projet repose sur une chaîne logicielle hétérogène couvrant la CAO, l'embarqué, et le *scripting* de bureau :

TABLE 3.2 – Chaîne logicielle utilisée.

Outil	Version	Usage
Onshape	Cloud 2024	CAO 3D paramétrique (pièces et assemblages)
Ultimaker Cura	5.x	Slicing pour impression 3D
Arduino IDE	2.3	Compilation du firmware C++ (cartes Nano)
Python	3.11	Interface graphique de bureau (PC)
Tkinter	built-in	Framework GUI Python
Selenium	4.x	Automatisation du navigateur (scraping Stellarium)
ChromeDriver	121.x	Pilote Selenium pour Chrome
PySerial	3.5	Communication série Python ↔ Arduino
Pillow (PIL)	10.x	Manipulation d'images (fond GUI)
AccelStepper	1.64	Bibliothèque Arduino pour step-pers (rampes)
LCD-I2C	2.3	Bibliothèque pour écran I2C
Stellarium Web	en ligne	Planétaire de référence pour le mode online
Launch Virtual Serial Port	9.x	Pont série virtuel pour debug

3.3 Le Corps du Télescope (Body)

3.3.1 L'Optique et ses Mathématiques

Cadre théorique : l'équation de conjugaison

La conception optique du télescope a débuté par une analyse rigoureuse fondée sur les lois de l'optique géométrique.

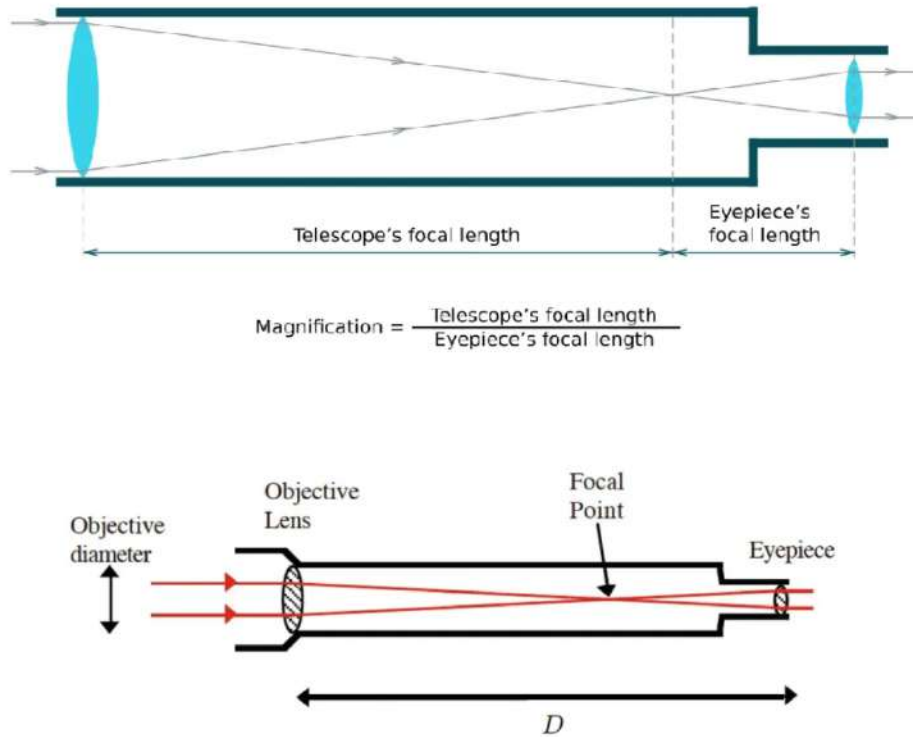


FIGURE 3.1 – Schéma des mathématiques optiques utilisées pour le dimensionnement

L'équation fondamentale qui régit le comportement d'une lentille mince dans l'approximation paraxiale est la *relation de conjugaison de Descartes* :

$$\frac{1}{p'} - \frac{1}{p} = \frac{1}{f} \quad (3.1)$$

où p est la position algébrique de l'objet (négatif s'il est en amont), p' celle de l'image, et f la distance focale de la lentille. Dans le cas astronomique où $p \rightarrow -\infty$ (l'objet est à l'infini), $1/p \rightarrow 0$ et l'image se forme exactement à $p' = f$: c'est le plan focal.

Vergence et dioptrie

Les lentilles que nous avons acquises auprès d'un opticien local sont caractérisées commercialement par leur **vergence** C (puissance optique), exprimée en dioptries (D, équivalent à m^{-1}) :

$$C = \frac{1}{f} \quad \text{où } f \text{ est en mètres.} \quad (3.2)$$

- **Objectif** : $C_{obj} = 1 \text{ D} \Rightarrow f_{obj} = \frac{1}{1} = 1 \text{ m} = 1000 \text{ mm}$.
- **Oculaire** : $C_{oc} = 12 \text{ D} \Rightarrow f_{oc} = \frac{1}{12} \approx 0,0833 \text{ m} \approx 83 \text{ mm}$.

Configuration képlérienne et grossissement

La configuration retenue est dite *képlérienne* : deux lentilles convergentes séparées d'une distance égale à la somme de leurs focales :

$$L_{\text{tube optique}} = f_{\text{obj}} + f_{\text{oc}} \approx 1000 + 83 = 1083 \text{ mm} \quad (3.3)$$

Le grossissement angulaire est défini comme le rapport de l'angle apparent α' sous lequel l'observateur voit l'image à l'angle réel α sous lequel l'objet est vu à l'œil nu :

$$G = \frac{\tan(\alpha')}{\tan(\alpha)} = \frac{f_{\text{obj}}}{f_{\text{oc}}} = \frac{1000}{83} \approx 12,05 \times \quad (3.4)$$

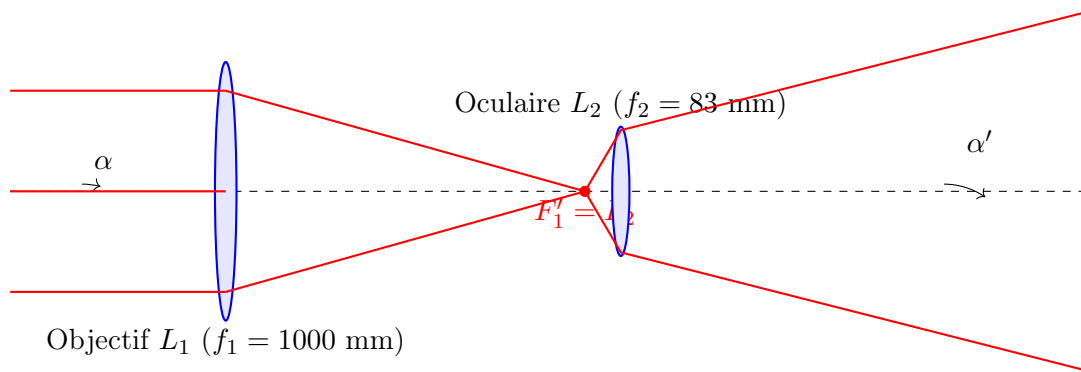


FIGURE 3.2 – Marche des rayons dans le télescope de Kepler : un faisceau parallèle (étoile à l'infini) entre sous un angle α , converge au foyer commun $F'_1 = F_2$, et ressort parallèle sous un angle $\alpha' = G \cdot \alpha$.

Discussion et choix de lentilles

Le grossissement modeste de $12\times$ a été choisi délibérément pour assurer un **champ de vision large** et lumineux, ce qui facilite considérablement le pointage initial des objets faibles et la phase de calibrage manuel. Augmenter le grossissement à $50\times$ ou $100\times$ aurait été techniquement possible avec un oculaire plus court (par exemple $20 \text{ mm} \Rightarrow 50\times$), mais aurait :

1. Rétréci le champ apparent (loi inversement proportionnelle).
2. Réduit la luminosité de l'image (proportionnelle à $1/G^2$).
3. Amplifié les vibrations mécaniques et les aberrations.

Le compromis $12\times$ se révèle adapté à un télescope de démonstration et de pointage avec un suivi GoTo non parfait. Une amélioration future consisterait à ajouter une **tourelle d'oculaires** interchangeables (par exemple 25 mm , 12 mm , 6 mm) pour offrir trois grossissements ($40\times$, $83\times$, $167\times$).

TABLE 3.3 – Bilan optique théorique du télescope.

Paramètre	Valeur
Focale objectif f_{obj}	1000 mm
Focale oculaire f_{oc}	83 mm
Vergence objectif	1 D
Vergence oculaire	12 D
Diamètre objectif D	≈ 50 mm
Rapport f/D	$f/20$
Grossissement G	$\approx 12\times$
Champ apparent oculaire	$\sim 50^\circ$
Champ réel	$\sim 4,2^\circ$

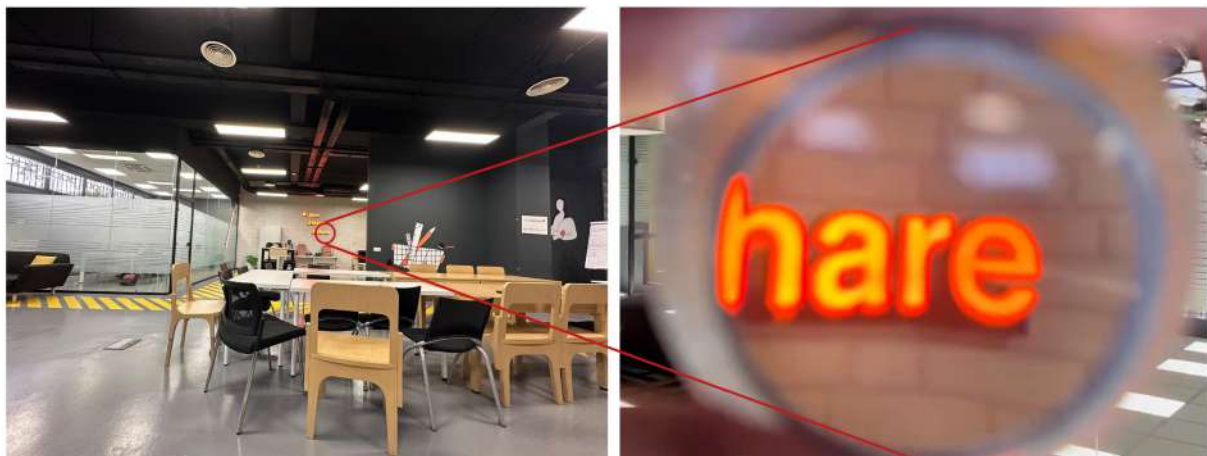


FIGURE 3.3 – Résultat du zoom obtenu

3.3.2 La Base Azimutale (Z-Axis)

Conception CAO

La base azimutale a été modélisée sur Onshape comme un assemblage paramétrique. Sa fonction est de permettre la rotation du télescope autour de l'axe vertical (azimut), tout en supportant le poids total de l'instrument (~ 5 kg) et en garantissant un mouvement sans frottement excessif. La structure se compose de deux plateaux circulaires en plexiglass de 5 mm d'épaisseur et de 250 mm de diamètre, séparés et guidés par un système tribologique optimisé.

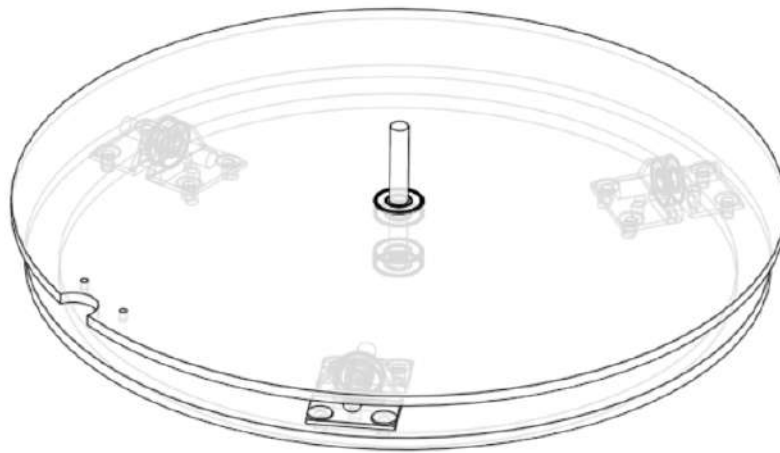


FIGURE 3.4 – Vue isométrique de la base azimutale modélisée sur Onshape.

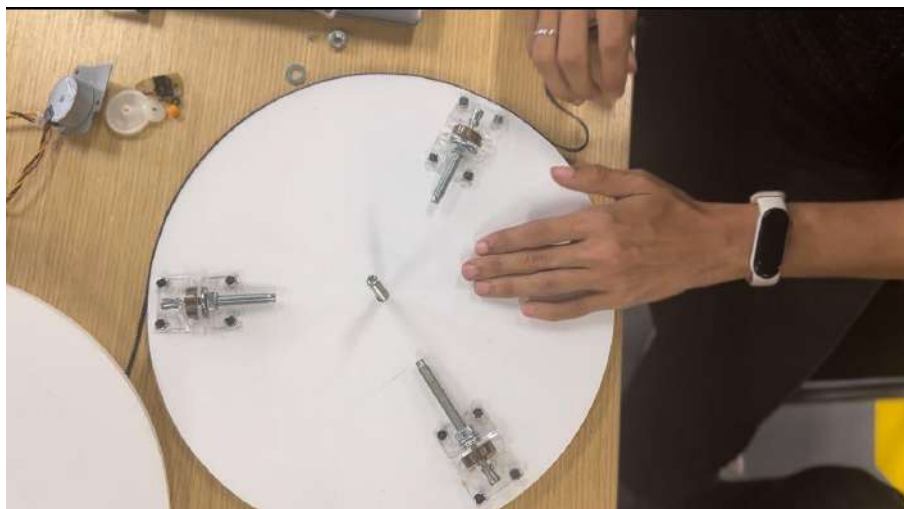


FIGURE 3.5 – Vue éclatée de la base azimutale révélant l’agencement des roulements, du pignon moteur et de la courroie crantée.

Description mécanique

La base est composée des éléments suivants :

- Deux plateaux en plexiglass découpés au laser ;
- Une **courroie crantée GT2** collée à la cyanoacrylate sur le périmètre du plateau inférieur, agissant ainsi comme une grande couronne dentée flexible ;
- Trois **roulements à billes 608ZZ** disposés à 120° les uns des autres, assurant la liaison glissière de révolution et la fluidité de rotation ;

- Un **roulement central** qui prend la charge axiale de l'instrument et garantit la coaxialité.
- Un **Moteur NEMA 17** qui fournira la force permettant de faire tourner le disque supérieur. ;

Calcul du rapport de réduction et résolution angulaire

La courroie crantée GT2 a un pas de 2 mm. Pour une circonférence du plateau de $\pi \cdot D_{\text{plat}} \approx \pi \cdot 250 \approx 785$ mm, le nombre total de dents de la couronne est :

$$N_{\text{couronne}} = \frac{785}{2} \approx 393 \text{ dents} \quad (3.5)$$

Le pignon moteur ayant $N_p = 11$ dents, le rapport de réduction est :

$$i_{Az} = \frac{N_{\text{couronne}}}{N_p} = \frac{393}{11} \approx 35,7 \quad (3.6)$$

La résolution angulaire azimutale, pour un moteur de $1,8^\circ/\text{pas}$ en microstepping $1/32$, devient :

$$\Delta\theta_{Az} = \frac{1,8^\circ}{32 \times i_{Az}} = \frac{1,8^\circ}{32 \times 35,7} \approx 1,58 \times 10^{-3}^\circ \approx 5,7'' \quad (3.7)$$

Cette résolution théorique est meilleure que le pouvoir séparateur de l'optique , garantissant un pointage limité par l'optique et non par la mécanique.

Assemblage et calibrage

L'assemblage suit un protocole précis : (1) découpe laser des plateaux, (2) impression 3D des supports de roulements, (3) collage de la courroie sur le plateau inférieur, (4) montage des trois roulements latéraux et du roulement central, (5) positionnement du moteur NEMA 17 et engrenement du pignon.



FIGURE 3.6 – la base azimutale assemblée.

3.3.3 L'Axe d'Altitude (X-Axis)

Conception CAO

L'axe d'altitude (rotation autour d'un axe horizontal) permet l'élévation du tube optique depuis l'horizon (0°) jusqu'au zénith (90°). Cet axe a été conçu autour d'un système d'engrenage en demi-cercle imprimé en 3D, contrairement à la base azimutale qui utilise une courroie. Ce choix est justifié par l'amplitude angulaire limitée requise (90°), ce qui permet l'utilisation d'une demi-couronne au lieu d'un système rotatif complet.

Par ailleurs, en raison d'une quantité limitée de plexiglas, la monture a été réalisée avec une hauteur réduite, ce qui a légèrement diminué la plage d'élévation du télescope. Toutefois, le système reste pleinement fonctionnel pour la majorité des observations astronomiques.

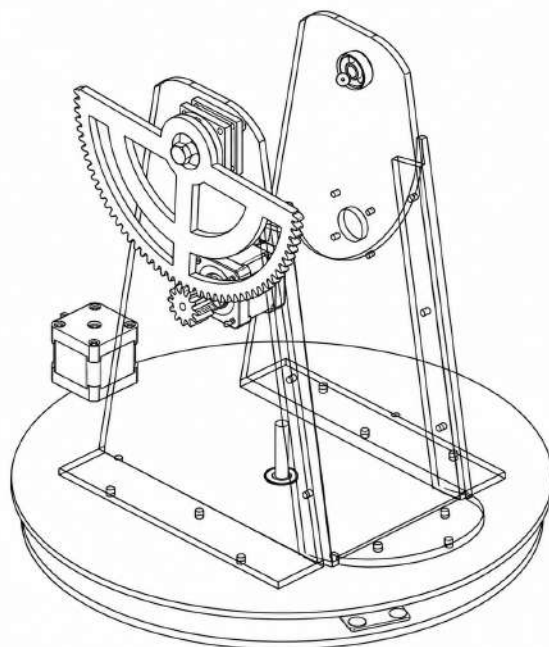


FIGURE 3.7 – Modélisation CAO de l'axe d'altitude sur Onshape.

Description mécanique

- Un **pignon de 11 dents** ($N_p = 11$) directement couplé à l'arbre du moteur NEMA 17;
- Une **demi-couronne de 96 dents** ($N_c = 96$) imprimée à moitié pour économiser du filament, fixée latéralement au tube optique;
- trois **plaques de plexiglass pliées à chaud** formant le bâti latéral (côté Y) entre la base azimutale et l'axe d'altitude.

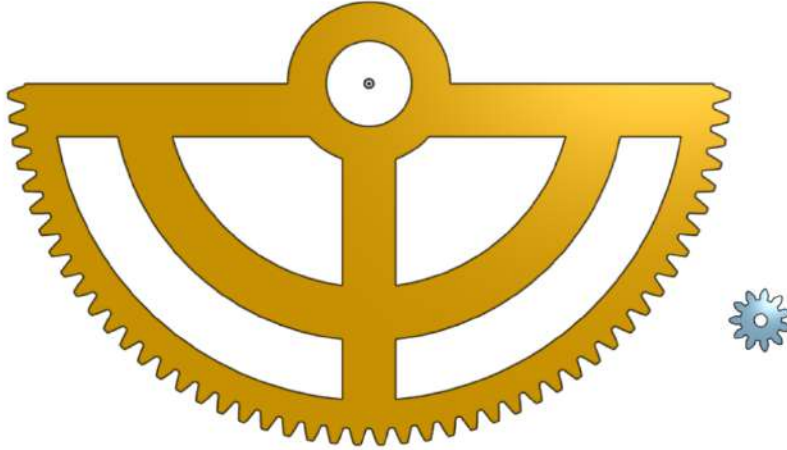


FIGURE 3.8 – demi-couronne de 96 dents et pignon de 11 dents

Calculs mécaniques

Le rapport d'engrenage est :

$$i_{Alt} = \frac{N_c}{N_p} = \frac{96}{11} \approx 8,727 \quad (3.8)$$

La résolution angulaire en altitude est donc :

$$\Delta\theta_{Alt} = \frac{1,8^\circ}{32 \times i_{Alt}} = \frac{1,8^\circ}{32 \times 8,727} \approx 6,45 \times 10^{-3}^\circ \approx 23,2'' \quad (3.9)$$

Le couple disponible à la sortie de la réduction est :

$$\tau_{Alt} = \tau_{moteur} \times i_{Alt} \times \eta_{engrenage} \approx 0,4 \times 8,727 \times 0,9 \approx 3,14 \text{ N} \cdot \text{m} \quad (3.10)$$

ce qui est largement suffisant pour vaincre le couple résistant lié au poids du tube ($\sim 0,3 \text{ N} \cdot \text{m}$ maximum à l'horizontale).

Pliage à chaud du plexiglass

Les plaques latérales du support ont été **pliées à chaud** à l'aide d'une résistance chauffante linéaire (*strip heater*) selon des arêtes pré-dessinées. Le PMMA (polyméthacrylate de méthyle) possède une température de transition vitreuse $T_g \approx 105^\circ\text{C}$; le pliage s'effectue typiquement à $130\text{-}160^\circ\text{C}$ pour obtenir une plastification homogène sans bullage.



FIGURE 3.9 – Pliage à chaud des plaques en plexiglass formant les supports latéraux.



FIGURE 3.10 – la base azimutale et altitude assemblée.

3.3.4 Le Corps Télescopique : tubes et pièces 3D

Tubes coulissants

Le corps optique du télescope est constitué de deux tubes en PVC concentriques, de diamètres extérieurs $\varnothing 63$ mm (tube extérieur portant l'objectif) et $\varnothing 50$ mm (tube intérieur portant l'oculaire). Le tube intérieur coulisse à l'intérieur du tube extérieur grâce à des **pièces de translation imprimées en 3D**, permettant ainsi la mise au point manuelle par variation continue de la distance objectif-oculaire autour du point théorique $L = f_{obj} + f_{oc} = 1083$ mm.

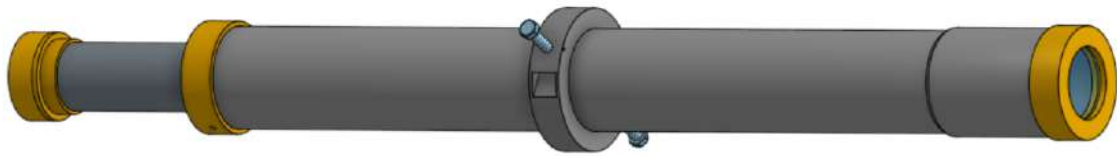


FIGURE 3.11 – Vue CAO des tubes télescopiques coulissants et de leurs pièces de raccordement imprimées en 3D.

Pièces imprimées en 3D

Les pièces 3D essentielles incluent :

- **Porte-objectif** : bague venant maintenir la lentille de 1 D dans l'extrémité du tube extérieur.
- **Porte-oculaire** : à l'extrémité opposée du tube intérieur.
- **Bagues de guidage** : deux bagues annulaires fendues qui assurent le centrage des tubes l'un dans l'autre.

Les paramètres d'impression utilisés étaient : matériau PLA, hauteur de couche 0,2 mm, remplissage 25%, 3 périmètres, vitesse 60 mm/s. Une tolérance d'ajustement de 0,3 mm a été ajoutée aux contacts mobiles pour compenser le retrait thermique.

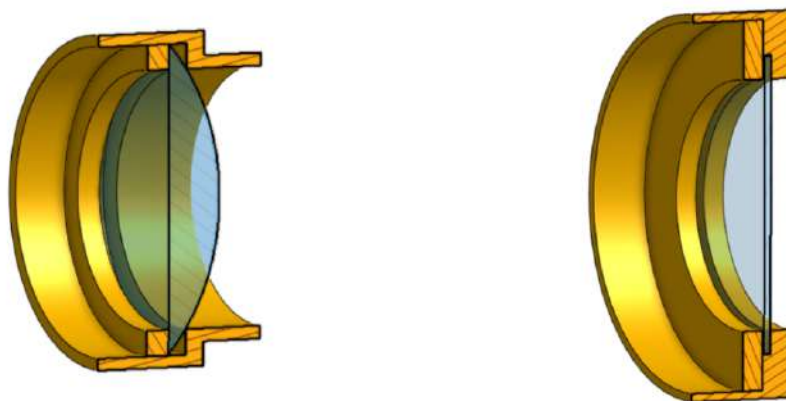


FIGURE 3.12 – Porte-objectif et oculaire

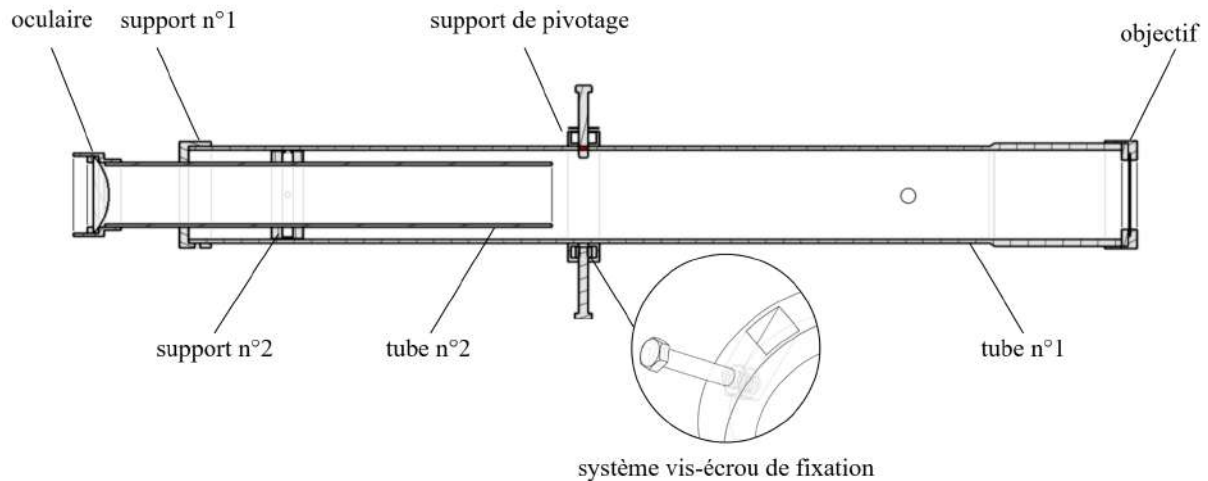


FIGURE 3.13 – vue interne du corps du télescope

3.3.5 Assemblage final du télescope

L'assemblage final intègre l'ensemble des sous-systèmes décrits ci-dessus : la base azimutale supporte les deux supports latéraux en plexiglass pliés, eux-mêmes portant l'axe d'altitude et la demi-couronne dentée. Le tube optique (objectif + oculaire avec leurs supports) est fixé à cet axe.



FIGURE 3.14 – Vue d'ensemble du télescope assemblé.

3.4 Les circuits :

3.4.1 Circuit 1 : Contrôle des moteurs

Architecture du circuit de puissance et de commande

Le circuit n°1 est l'étage de puissance et de contrôle des deux moteurs pas-à-pas. Il s'articule autour d'un Arduino Nano (le « cerveau moteur ») qui pilote deux drivers A4988 via leurs broches STEP et DIR. L'alimentation 12 V traverse un fusible 4 A de protection contre les courts-circuits.

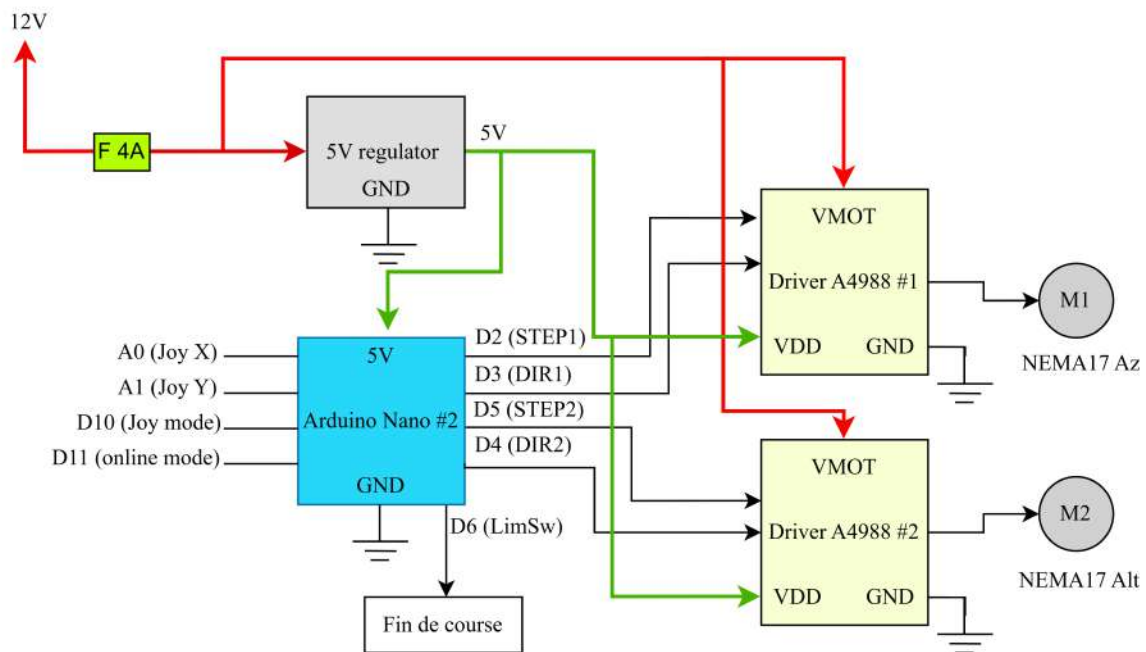


FIGURE 3.15 – Schéma synoptique simplifié du circuit n°1 (contrôle des moteurs)

Microstepping et précision

Les drivers A4988 sont réglés sur un microstepping de 1/32 via les broches MS1, MS2, MS3 mises à HIGH plus deux condensateurs pour assurer une tension stable pour eux, **voir l'annexe pour le circuit détaillé.**

Ce mode subdivise chaque pas mécanique du moteur en 32 micro-pas, ce qui :

1. Augmente la résolution angulaire d'un facteur 32 ;
2. Lisse les transitions, réduisant les vibrations et le bruit acoustique ;

TABLE 3.4 – Affectation des broches de l’Arduino Nano n°2 (contrôle moteurs).

Broche	Fonction	Description
D2	STEP1	Impulsions pas moteur Az
D3	DIR1	Sens de rotation moteur Az
D4	DIR2	Sens de rotation moteur Alt
D5	STEP2	Impulsions pas moteur Alt
D6	LIMIT_SW	Fin de course (homing)
D10	PIN_JOY	Indication mode joystick (voir les parties suivantes)
D11	PIN_ONLINE	Indication mode en ligne (voir les parties suivantes)
A0	JOY_X	Lecture analogique X
A1	JOY_Y	Lecture analogique Y
RX/TX	UART	Liaison série vers Arduino n°1

3.4.2 Circuit 2 : la télécommande

Le Boîtier de Commande (Remote Controller)

Justification de l’architecture à deux Arduino Architecture de la télécommande et de la communication

Le système de contrôle du télescope nécessite une interface intuitive permettant à l’opérateur de basculer entre les modes de fonctionnement, de configurer les paramètres, d’ajuster manuellement le pointage via un joystick et de sélectionner des objets célestes à partir d’un catalogue.

Dans un premier temps, ces fonctions avaient été intégrées directement sur la carte de contrôle principale. Cependant, au cours du développement, il est apparu que la centralisation à la fois de l’interface utilisateur et de la charge de calcul sur un seul microcontrôleur n’était pas une approche viable. En effet, l’Arduino principal était déjà sollicité par des traitements mathématiques intensifs transformations de coordonnées, calculs de pointage et contrôle en temps réel des moteurs ne laissant pas suffisamment de ressources pour gérer les entrées utilisateur de manière fluide et fiable.

Pour résoudre ce problème, un second Arduino (**arduino n°1**) a été introduit, dédié exclusivement à l’interface homme-machine et aux traitements mathématiques. Cette unité prend en charge les entrées du joystick, les boutons de navigation ainsi que l’afficheur LCD 16×2 fournissant un retour en temps réel à l’opérateur. Il agit comme un processeur frontal : il capte les intentions de l’utilisateur, effectue les calculs de précision nécessaires, et transmet les commandes résultantes à l’Arduino

principal, lequel reste entièrement dédié à la gestion de la puissance et à la commande des moteurs.

Les deux Arduinos communiquent via une liaison série UART. Afin de garantir l'intégrité de cette communication, un isolateur numérique (ISO7221) est placé sur les lignes UART côté carte principale. La télécommande est alimentée par une batterie indépendante, tandis que la carte principale est alimentée par l'alimentation centrale. Ainsi, les deux masses ne sont jamais reliées électriquement : malgré une communication continue entre les deux Arduinos, ils restent électriquement isolés.



FIGURE 3.16 – isolateur numérique (ISO7221)

Cette isolation permet d'empêcher la propagation des parasites, fluctuations de masse et transitoires de commutation générés par les drivers de moteurs pas-à-pas A4988 vers l'électronique de la télécommande. Elle protège ainsi à la fois la communication UART et le bus I2C de l'écran LCD, particulièrement sensible aux perturbations et pouvant se bloquer complètement à la suite d'un seul bit corrompu.

- (a) Les A4988 hachent le courant moteur à environ 50 kHz, créant des transitoires de tension sur la masse capables de fausser la lecture analogique du joystick (10 bits, ~ 5 mV de résolution) ;
- (b) Le bus I2C de l'écran LCD est très sensible aux glitches : un seul bit erroné suffit à figer l'affichage ou à corrompre la communication.

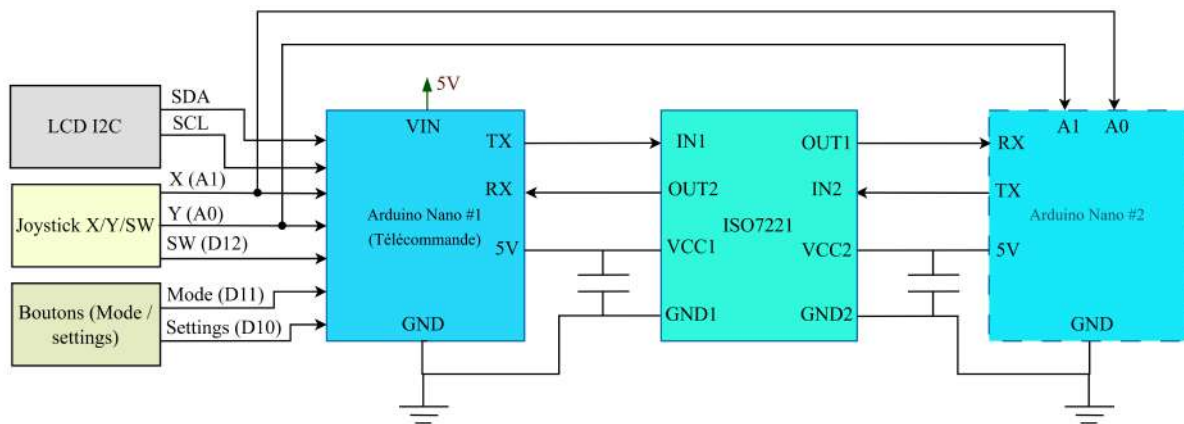


FIGURE 3.17 – Schéma synoptique simplifié du circuit n°2 (Télécommande)

L'Arduino Nano n°1 intègre :

- Un écran **LCD I2C 16×2** (adresse 0x27) via les broches A4/A5 (SDA/SCL) ;
- Un **joystick analogique** avec axes X (A1) et Y (A0), et bouton-poussoir intégré (D12) ;
- Deux **boutons-poussoirs** : « Mode » (D11) pour basculer entre Normal / Réglage, et « Scope » (D10) pour basculer entre les sous-modes Offline / Online / Joystick ;
- Deux **sorties logiques** : D8 (mode en ligne) et D9 (mode joystick) routées vers l'Arduino n°2, **Ces éléments ne sont pas représentés ici ; se référer à l'annexe pour les schémas détaillés.**

CAO et impression du boîtier de la télécommande

Le boîtier de télécommande a été modélisé sur Onshape en deux demi-coques s'assemblant par quatre vis M3. Les découpes en façade épousent précisément l'écran LCD, le joystick et les boutons. Un passe-câble à l'arrière laisse sortir le câble blindé de 8 conducteurs vers le télescope.



FIGURE 3.18 – Modélisation CAO du boîtier de télécommande.



FIGURE 3.19 – Boîtier de télécommande finalisé, imprimé en PLA.

3.5 Programmation et Modes de Fonctionnement

3.5.1 Organigramme général du système

La programmation embarquée et logicielle du télescope orchestre l'ensemble des sous-systèmes décrits précédemment. L'organigramme global du firmware du télescope est présenté ci-dessous :

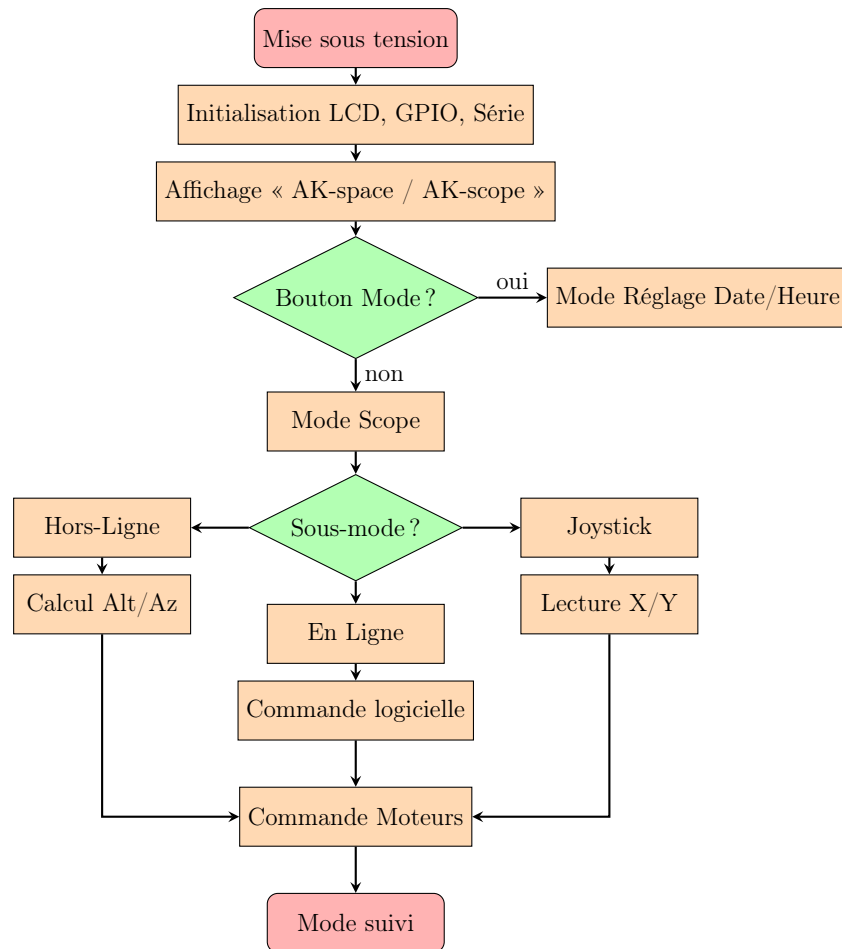


FIGURE 3.20 – Organigramme global du firmware de télescope.

3.5.2 Le Mode Hors-Ligne (Offline) : Astronomie Sphérique

Remarque

Algorithme astronomique Ce mode garantit l'autonomie du télescope en "pleine brousse", sans PC. Le microcontrôleur embarque les coordonnées équatoriales (RA, Dec) d'un catalogue de 20 étoiles. Les équations de la trigonométrie sphérique sont encodées pour convertir ces coordonnées absolues en coordonnées horizontales (Alt, Az) au point de l'observateur.

Représentation de la sphère céleste et triangle de position

Pour bien comprendre les équations qui suivent, il est indispensable de visualiser la **sphère céleste**, sphère idéale de rayon infini sur laquelle sont projetés les astres. Deux systèmes de coordonnées coexistent :

- Le système **équatorial** (RA, Dec), lié au plan équatorial terrestre prolongé. L'ascension droite RA mesure l'angle dans le plan équatorial à partir du point

vernal (γ), la déclinaison Dec mesure la latitude céleste à partir de l'équateur. **ce système est universel et indépendant de la position de l'observateur sur Terre**

- Le système **horizontal** (Alt, Az), lié à l'observateur. L'altitude Alt mesure la hauteur au-dessus de l'horizon, l'azimut Az se compte depuis le Nord en allant vers l'Est. Ce système est *local*.

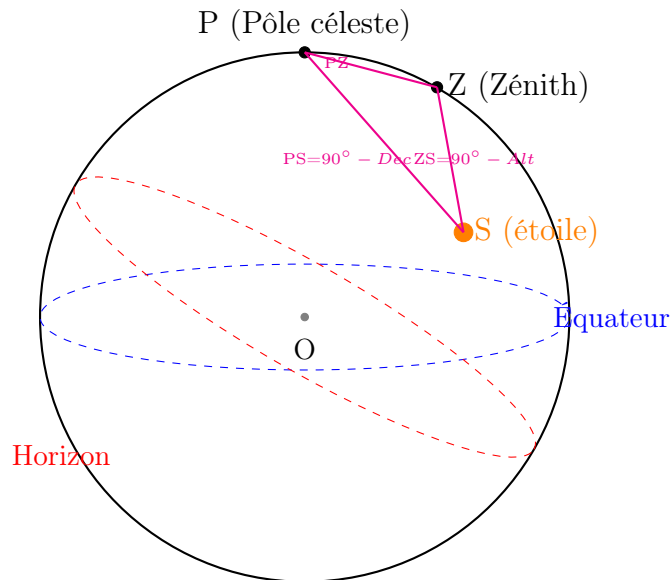


FIGURE 3.21 – Sphère céleste et triangle de position sphérique PZS liant le pôle céleste P , le zénith Z et l'étoile S . Le côté $PZ = 90^\circ - \phi$ (colatitude), $PS = 90^\circ - Dec$ (distance polaire), $ZS = 90^\circ - Alt$ (distance zénithale). L'angle au sommet P est l'angle horaire HA .

L'application des **formules fondamentales de la trigonométrie sphérique** (analogues de la loi des cosinus et des sinus) au triangle PZS permet de dériver directement les équations de passage entre les deux systèmes, présentées ci-dessous.

1. Temps Universel (UT) et Période Julienne (d) : L'heure locale (h, m, s) est décimalisée :

$$UT = h + \frac{m}{60} + \frac{s}{3600} \text{ (heures)} \quad (3.11)$$

Le nombre de jours d depuis l'époque J2000 est calculé en tenant compte des bissextiles :

$$d = 367 \cdot Y - \left\lfloor \frac{7 \cdot (Y + \lfloor \frac{M+9}{12} \rfloor)}{4} \right\rfloor + \left\lfloor \frac{275 \cdot M}{9} \right\rfloor + D - 730530 \quad (3.12)$$

2. Le Temps Sidéral : L'angle de l'équinoxe de printemps (GST) est déterminé puis ajusté par la Longitude locale (en degrés divisés par 15) pour obtenir le LST :

$$GST = 6.697374558 + 0.0657098244 \cdot d + 1.0027379 \cdot UT \pmod{24} \quad (3.13)$$

$$LST = GST + \frac{\text{Longitude}}{15} \pmod{24} \quad (3.14)$$

3. L'Angle Horaire (HA) et Matrice de Transformation : L'angle horaire définit la position de l'astre par rapport au méridien :

$$HA = (LST - RA) \times 15 \pmod{360} \quad (\text{en degrés}) \quad (3.15)$$

La projection sphérique donne finalement l'Altitude et l'Azimut de la cible :

$$\sin(Alt) = \sin(Dec) \sin(Lat) + \cos(Dec) \cos(Lat) \cos(HA) \quad (3.16)$$

$$\cos(Az) = \frac{\sin(Dec) - \sin(Alt) \sin(Lat)}{\cos(Alt) \cos(Lat)} \quad (3.17)$$

L'Aзимut est rectifié à $Az = 360^\circ - Az$ si $\sin(HA) > 0$.

Dérivation des équations de transformation

Les deux équations (3.16) et (3.17) ne sont pas postulées : elles découlent directement de l'application de la *formule fondamentale de la trigonométrie sphérique* au triangle PZS .

Loi des cosinus sphérique : pour un triangle sphérique de côtés a, b, c et d'angles opposés A, B, C , on a :

$$\cos(a) = \cos(b) \cos(c) + \sin(b) \sin(c) \cos(A) \quad (3.18)$$

En posant $a = ZS = 90^\circ - Alt$, $b = PZ = 90^\circ - \phi$, $c = PS = 90^\circ - Dec$ et $A = HA$ (angle au pôle), il vient :

$$\cos(90^\circ - Alt) = \cos(90^\circ - \phi) \cos(90^\circ - Dec) + \sin(90^\circ - \phi) \sin(90^\circ - Dec) \cos(HA) \quad (3.19)$$

$$\sin(Alt) = \sin(\phi) \sin(Dec) + \cos(\phi) \cos(Dec) \cos(HA) \quad (3.20)$$

ce qui démontre l'équation (3.16). De même, en appliquant la loi des cosinus sur le côté PS , on obtient l'équation (3.17).

Catalogue d'étoiles embarqué

Le firmware Arduino embarque les coordonnées équatoriales de 20 étoiles brillantes. Ces coordonnées (ascension droite et déclinaison) constituent un catalogue de référence supposé fixe à l'échelle humaine, car elles décrivent la position des étoiles dans le référentiel céleste. Cependant, en réalité, ces valeurs évoluent lentement au cours du temps en raison de la précession des équinoxes et du mouvement propre des étoiles.

Dans notre cas, ce catalogue reste suffisamment précis pour garantir un pointage correct sur une durée d'environ 10 à 15 ans. Au-delà de cette période, une mise à jour des coordonnées devient nécessaire afin de conserver la précision des calculs et la cohérence avec l'époque astronomique de référence. Le tableau 3.5 donne la liste complète.

TABLE 3.5 – Catalogue des 20 étoiles embarquées dans le firmware.

Étoile	RA (h)	Dec (°)	Constellation
Sirius	6,7525	-16,7161	Grand Chien
Canopus	6,3992	-52,6957	Carène
Arcturus	14,2610	+19,1825	Bouvier
Vega	18,6156	+38,7837	Lyre
Capella	5,2782	+45,9979	Cocher
Rigel	5,2423	-8,2016	Orion
Procyon	7,6550	+5,2250	Petit Chien
Achernar	1,6286	-57,2368	Éridan
Betelgeuse	5,9195	+7,4071	Orion
Hadar	14,0637	-60,3729	Centaure
Altaïr	19,8463	+8,8683	Aigle
Aldebaran	4,5987	+16,5092	Taureau
Antarès	16,4901	-26,4320	Scorpion
Spica	13,4199	-11,1613	Vierge
Pollux	7,7553	+28,0262	Gémeaux
Fomalhaut	22,9608	-29,6222	Poisson Austral
Deneb	20,6905	+45,2803	Cygne
Régulus	10,1395	+11,9672	Lion
Castor	7,5766	+31,8884	Gémeaux
Alnaïr	22,1372	-46,9601	Grue

Organigramme du mode hors-ligne

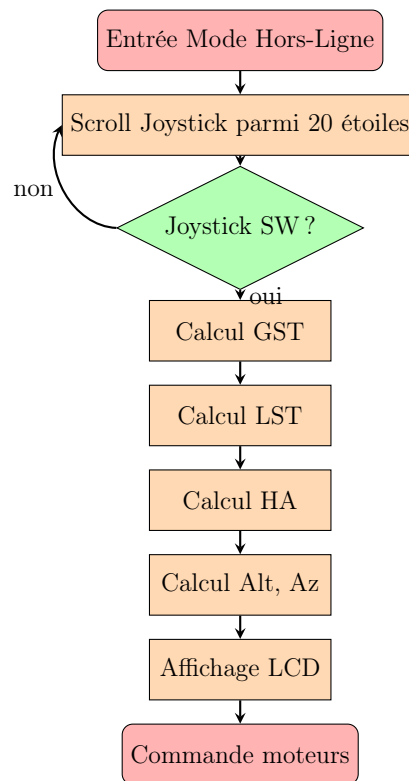


FIGURE 3.22 – Organigramme du mode hors-ligne.

Extrait de code Arduino : fonctions de calcul

```

1 float calculateGST(int year, int month, int day, int hour, int
  minute, int second) {
2   float UT = hour - 7 + minute / 60.0 + second / 3600.0;
3   int yTemp = year + (month + 9) / 12;
4   float d = 367 * year - (7 * yTemp) / 4 + (275 * month) / 9 + day
  - 730530;
5   float GST = 6.697374558 + 0.06570982441908 * d + 1.00273790935 *
  UT;
6   GST = fmod(GST, 24.0);
7   return GST;
8 }
9
10 float calculateLST(float gst, float longitude) {
11   float LST = gst + longitude / 15.0;
12   LST = fmod(LST, 24.0);
13   return LST;
14 }
15
16 void calculateAltAz(float ra, float dec, float lat, float lst,
17   float &alt, float &az) {

```

```

18 float HA = fmod((lst * 15.0 - ra * 15.0), 360.0);
19 if (HA < 0) HA += 360.0;
20 HA = degreesToRadians(HA);
21 dec = degreesToRadians(dec);
22 lat = degreesToRadians(lat);
23 float sinAlt = sin(dec)*sin(lat) + cos(dec)*cos(lat)*cos(HA);
24 alt = asin(sinAlt);
25 float cosAz = (sin(dec) - sin(alt)*sin(lat)) / (cos(alt)*cos(lat)
26 );
27 cosAz = constrain(cosAz, -1, 1);
28 az = acos(cosAz);
29 alt = radiansToDegrees(alt);
30 az = radiansToDegrees(az);
31 if (sin(HA) > 0) az = 360.0 - az;
32 }

```

Listing 3.1 – Fonctions calculateGST, calculateLST et calculateAltAz.

Cas d'Étude Pratique : Pointage de l'étoile Vega

Considérons l'étoile Capella ($RA = 5,2782$ h, $Dec = +45,9979^\circ$) observée depuis Agadir ($Lat = 30,4^\circ N$, $Long = -9,6^\circ$). Pour le jour $d = 10995$, le calcul donne une Altitude $Alt \approx 73,97^\circ$ et un Azimut $Az \approx 347,99^\circ$.

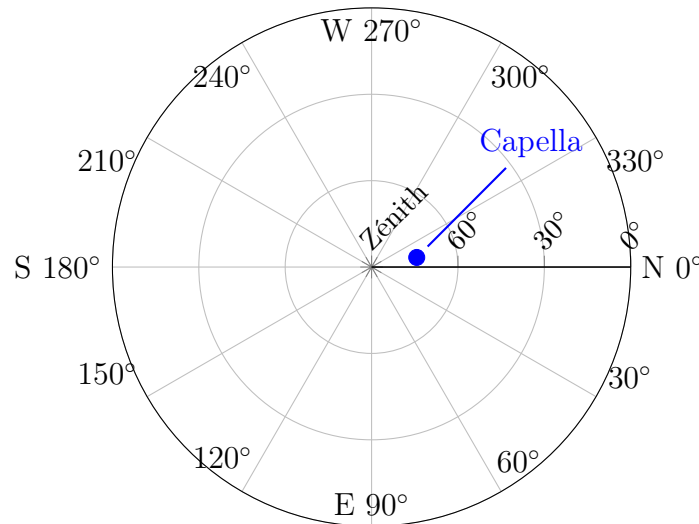


FIGURE 3.23 – Représentation polaire locale (Azimut, Altitude radiale vers le centre) de la cible Capella. $Alt \approx 73,97^\circ$, $Az \approx 347,99^\circ$, observée depuis Agadir ($Lat = 30,4^\circ N$, $Long = -9,6^\circ$) pour $d = 10995$.

3.5.3 Le Mode En Ligne (Online) : Intégration Python & Stellarium

Architecture logicielle

Le mode en ligne offre l'accès à une base de données quasi infinie (plus de 600 000 objets célestes) en exploitant le **planétaire Stellarium Web**. La chaîne logicielle complète est la suivante :

- L'utilisateur ouvre l'interface graphique Python sur son PC ;
- Il sélectionne un port série actif (scan automatique des ports COM1 à COM256) et un objet céleste dans une liste de 50 cibles ;
- Un *thread* dédié lance ChromeDriver (Selenium) en mode **headless**, charge <https://stellarium-web.org/>, ferme la pop-up de bienvenue, saisit le nom de l'objet dans la barre de recherche, et clique sur le résultat ;
- Les éléments DOM de classe **radecVal** contenant les coordonnées horizontales (degrés, minutes, secondes) sont extraits ;
- Le résultat formaté est envoyé par UART (115200 baud) à l'Arduino n°2 qui le decode et commande les moteurs.

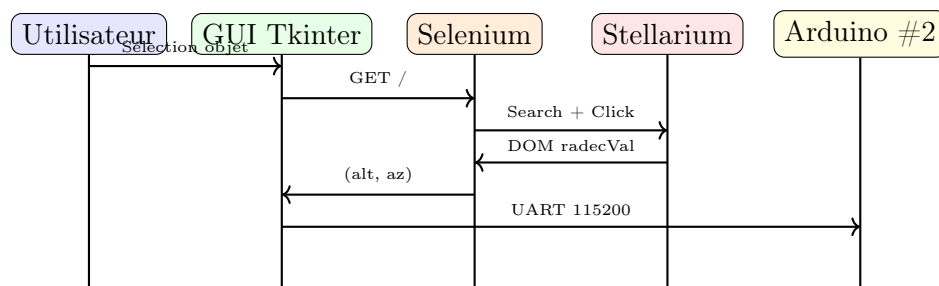


FIGURE 3.24 – Diagramme de séquence du mode en ligne (Online).

Interface graphique Tkinter

L'interface a été conçue avec Tkinter et utilise un fond animé (la Voie Lactée) défilant lentement pour une expérience utilisateur immersive. Les widgets principaux sont :

- Une Combobox listant les ports série détectés (`serial_ports()`) ;
- Une Combobox listant les 50 objets célestes prédéfinis ;
- Un Button « Search » déclenchant la recherche dans un thread séparé pour ne pas figer l'interface ;
- Une ScrolledText affichant les coordonnées résultantes.

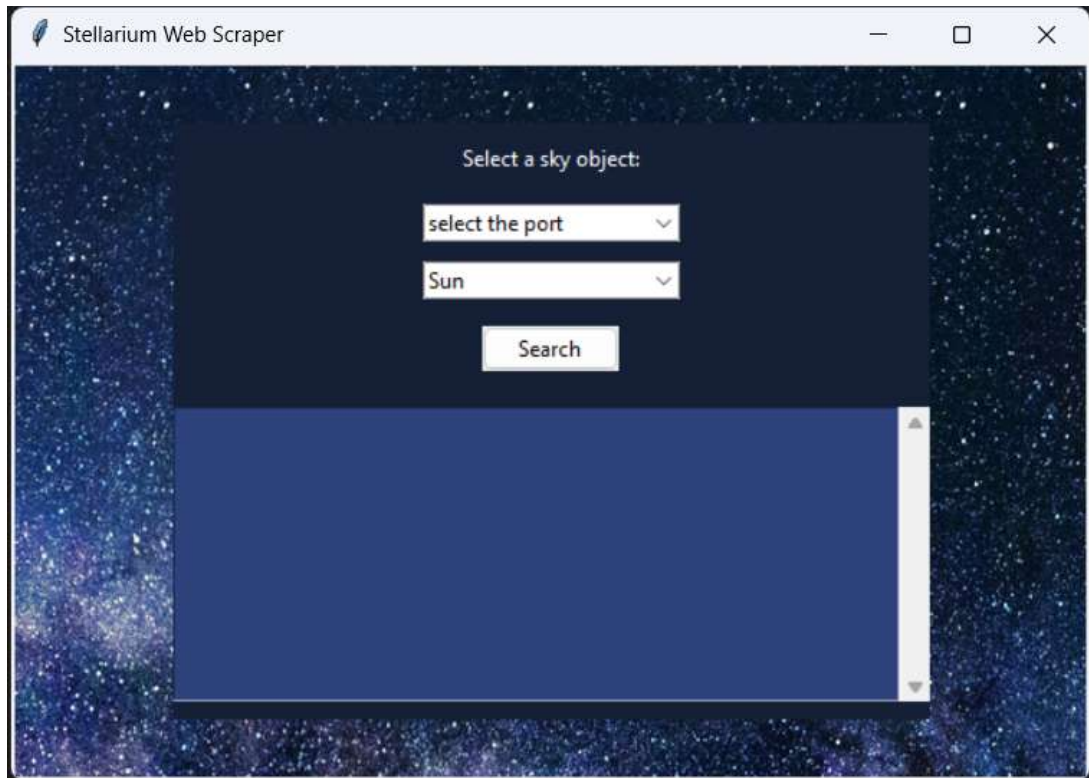


FIGURE 3.25 – Capture d’écran de l’interface Python (Tkinter).

Extraction Selenium des angles

```

1 radec_vals = driver.find_elements(By.CLASS_NAME, "radecVal")
2 # Indices 6-8 : altitude (deg, min, sec)
3 alt_angle = radec_vals[6].text.strip()
4 alt_min   = radec_vals[7].text.strip()
5 alt_sec   = radec_vals[8].text.strip()
6 # Indices 9-11 : azimut (deg, min, sec)
7 az_angle  = radec_vals[9].text.strip()
8 az_min    = radec_vals[10].text.strip()
9 az_sec    = radec_vals[11].text.strip()
10 # Formatage et envoi s rie
11 result = f"{az_angle} {az_min} {az_sec}\n{alt_angle} {alt_min} {
    alt_sec}\n"
12 arduino.write(bytes(result, 'utf-8'))
13 time.sleep(0.05)

```

Listing 3.2 – Extraction Selenium de l’Altitude et de l’Azimut.

Analyse ligne par ligne du code Python

Le code Python comporte des fonctions clés que nous analysons ici (puisque le source original est peu commenté) :

- `serial_ports()` : itère sur les noms COM1 à COM256 (sous Windows), tente de les ouvrir via `serial.Serial(port)` et conserve uniquement ceux disponibles. Robuste face aux ports verrouillés ou inexistants grâce à un bloc `try/except (OSError, SerialException)`.
- `create_widgets()` : assemble les widgets Tkinter dans un `Frame` centré, avec un style sombre (fonds bleu marine #151f36).
- `start_search()` : lance `search_object()` dans un `Thread` daemon pour préserver la réactivité de l'interface.
- `search_object()` : (1) configure `ChromeDriver` headless, (2) attend que le DOM soit chargé via `WebDriverWait`, (3) ferme la pop-up de bienvenue si elle apparaît, (4) saisit le nom de l'objet et envoie Enter, (5) clique sur l'élément retourné, (6) parse les `radecVal`, (7) envoie le résultat à l'Arduino, (8) ferme proprement le driver dans un bloc `finally`.
- `set_background_image()` : charge une image PNG de la Voie Lactée via `PIL (Image.open)`, la redimensionne en 1200×800 avec un échantillonnage LANCZOS (haute qualité), et la dessine sur un `Canvas Tkinter`.
- `animate_background()` : déplace l'image de fond de 0,5 px à chaque appel et se réinvoque toutes les 50 ms via `master.after(50, ...)`.

exemple

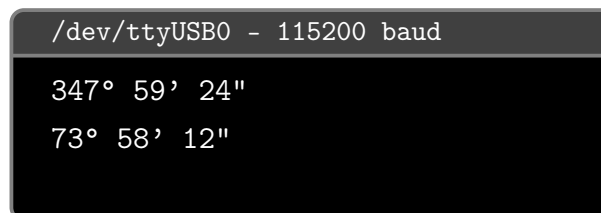


FIGURE 3.26 – Trame série envoyée à l'Arduino pour l'étoile Capella (Azimut : 34759'24'', Altitude : 7358'12'').

Décodage côté Arduino

Côté Arduino n°2, la chaîne reçue par UART est décodée par `inputString.indexOf('°')`, `indexOf('\'')` et `indexOf('\\"')` pour extraire degré, minute et seconde de chaque angle. La conversion en degrés décimaux et le calcul du delta par rapport à la position actuelle alimentent la bibliothèque `AccelStepper` qui génère des rampes d'accélération douces vers la consigne.

3.5.4 Le Mode de Suivi (Following Mode) - Ajout Algorithmique

Motivation et principe

Une fois la cible acquise par le mode hors-ligne ou en ligne, la rotation de la Terre déplace l'objet apparent dans le ciel à la **vitesse sidérale** :

$$\omega_{\text{sidérale}} = \frac{d(HA)}{dt} = \frac{360^\circ}{86164,1 \text{ s}} \approx 15,04107^\circ/\text{heure} \approx 4,178 \times 10^{-3} \text{ }^\circ/\text{s} \quad (3.21)$$

Le **mode de suivi** (*Following Mode*) compense cette dérive en réactualisant en continu les coordonnées horizontales de l'objet, indispensable pour deux usages :

- (a) **Observation visuelle prolongée** : maintenir l'objet centré dans l'oculaire sans intervention manuelle ;
- (b) **Astrophotographie par stacking** : compenser le filé d'étoiles sur des poses unitaires de 10 s à 1 min, puis empiler N poses pour amplifier le rapport signal sur bruit

Dérivation des vitesses angulaires

Sur une monture Dobsonienne, contrairement à une monture équatoriale qui ne nécessite qu'une rotation à vitesse constante autour d'un axe, l'Altitude et l'Azimut varient *simultanément* et de manière **non linéaire**. En dérivant les équations (3.16) et (3.17) par rapport au temps (règle de composition, avec ϕ et Dec constants car la latitude et la déclinaison ne dépendent pas du temps), on obtient les vitesses angulaires requises sur chaque moteur :

Vitesse en altitude. Dérivons $\sin(Alt) = \sin(\phi) \sin(Dec) + \cos(\phi) \cos(Dec) \cos(HA)$:

$$\cos(Alt) \cdot \frac{d(Alt)}{dt} = -\cos(\phi) \cos(Dec) \sin(HA) \cdot \frac{d(HA)}{dt} \quad (3.22)$$

En remarquant que $\cos(\phi) \cos(Dec) \sin(HA) = \cos(Alt) \sin(Az)$ (autre identité du triangle sphérique), il vient :

$$\boxed{v_{Alt}(t) = \frac{d(Alt)}{dt} = -\sin(Az) \cdot \omega_{\text{sidérale}}} \quad (3.23)$$

Vitesse en azimuth. En dérivant l'identité $\cos(Az) = \frac{\sin(Dec) - \sin(Alt) \sin(\phi)}{\cos(Alt) \cos(\phi)}$, et après simplification :

$$\boxed{v_{Az}(t) = \frac{d(Az)}{dt} = (\sin(\phi) + \cos(\phi) \cos(Az) \tan(Alt)) \cdot \omega_{\text{sidérale}}} \quad (3.24)$$

Cas particuliers et singularité au zénith

- Lorsque l’astre traverse le méridien ($HA = 0$), $\sin(Az) = 0$ et donc $v_{Alt} = 0$: le mouvement est purement azimutal.
- Lorsque l’astre passe au zénith ($Alt \rightarrow 90^\circ$), $\tan(Alt) \rightarrow +\infty$ et v_{Az} diverge : c’est la **singularité au zénith**, caractéristique des montures alt-azimutales. Cette zone d’environ $\pm 2^\circ$ autour du zénith est inaccessible au suivi.

Note : C’est dans le cas général. Dans le cadre de notre réalisation, une contrainte mécanique supplémentaire s’impose : l’altitude du télescope ne doit pas dépasser $Alt_{\max} = 50^\circ$. Toute cible dont l’altitude calculée excède ce seuil doit être rejetée avant l’envoi des commandes aux moteurs.

Discrétisation et conversion en pas moteur

Le firmware échantillonne les équations (3.23) et (3.24) à intervalle $\Delta t = 1$ s grâce à un timer matériel (Timer1 de l’ATmega328). À chaque tick :

$$\Delta\theta_{Alt} = v_{Alt}(t) \cdot \Delta t, \quad \Delta\theta_{Az} = v_{Az}(t) \cdot \Delta t \quad (3.25)$$

On convertit ensuite en micro-pas via les rapports de réduction calculés précédemment :

$$N_{\mu\text{-pas, Alt}} = \frac{\Delta\theta_{Alt}}{1,8^\circ} \times 32 \times i_{Alt} \quad (3.26)$$

$$N_{\mu\text{-pas, Az}} = \frac{\Delta\theta_{Az}}{1,8^\circ} \times 32 \times i_{Az} \quad (3.27)$$

La partie de code responsable du suivi

```

1 volatile bool tick = false;
2 ISR(TIMER1_COMPA_vect) { tick = true; } // interruption 1 Hz
3
4 #define ALT_MAX 50.0 // Limitation mécanique du système
5
6 void followingLoop() {
7     if (tick) {
8         tick = false;
9         // 1. Recalcul du temps sidéral
10        incrementClock(); // +1 s sur horloge interne
11        float gst = calculateGST(year, month, day, hour, minute, second);
12        float lst = calculateLST(gst, longitude);
13        // 2. Recalcul Alt/Az
14        float newAlt, newAz;
```

```
15     calculateAltAz(ra, dec, lat, lst, newAlt, newAz);
16     // 3. V rification limitation m canique (Alt <= 50 )
17     if (newAlt > ALT_MAX) {
18         lcd.clear();
19         lcd.print("Alt > 50deg !");
20         lcd.setCursor(0, 1);
21         lcd.print("Cible inaccessible");
22         motor1.stop();
23         motor2.stop();
24         return; // sortir sans bouger les moteurs
25     }
26     // 4. Delta par rapport a la position precedente
27     float dAlt = newAlt - prealt;
28     float dAz  = newAz  - preaz;
29     // Correction du passage 360
30     if (dAz > 180) dAz -= 360;
31     if (dAz < -180) dAz += 360;
32     // 5. Conversion en micro-pas
33     long stepsAlt = (long)((dAlt / 1.8) * 32 * i_Alt);
34     long stepsAz  = (long)((dAz / 1.8) * 32 * i_Az);
35     // 6. Commande differentielle (rampe douce)
36     motor1.move(stepsAlt);
37     motor2.move(stepsAz);
38     prealt = newAlt;
39     preaz  = newAz;
40 }
41 motor1.run();
42 motor2.run();
43 }
```

Listing 3.3 – La partie de code responsable du suivi

Organigramme du Following Mode

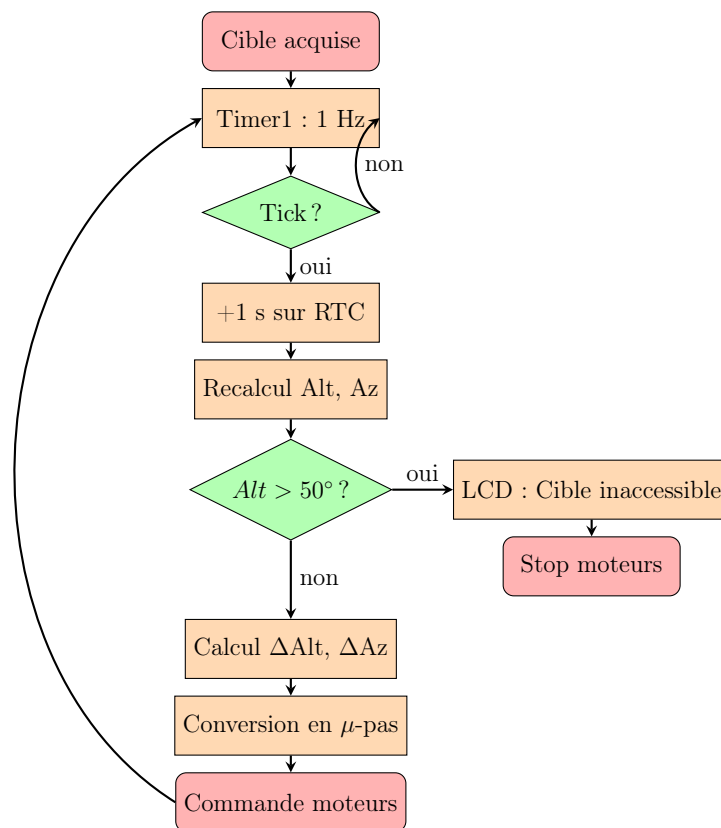


FIGURE 3.27 – Organigramme du mode de suivi (Following Mode)

Simulation de la trajectoire de Capella

Pour valider l'algorithme, nous avons simulé numériquement la trajectoire apparente de Capella observée depuis Agadir sur une période allant de 18h00 à 09h00 UT. Les coordonnées calculées sont reportées sur le diagramme polaire ci-dessous. Afin de vérifier l'exactitude des résultats obtenus, les positions calculées ont été comparées à celles fournies par le logiciel Stellarium. Cette comparaison a confirmé la cohérence des calculs et la validité de l'algorithme de conversion et de suivi développé

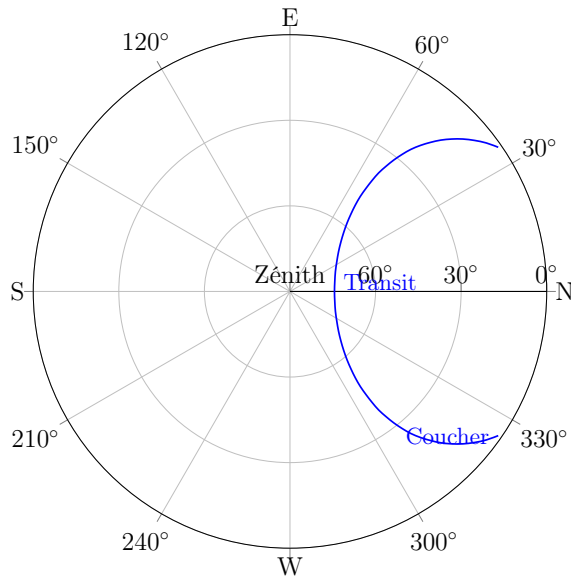


FIGURE 3.28 – Trajectoire complète de Capella depuis Agadir (latitude $30,4^\circ\text{N}$).

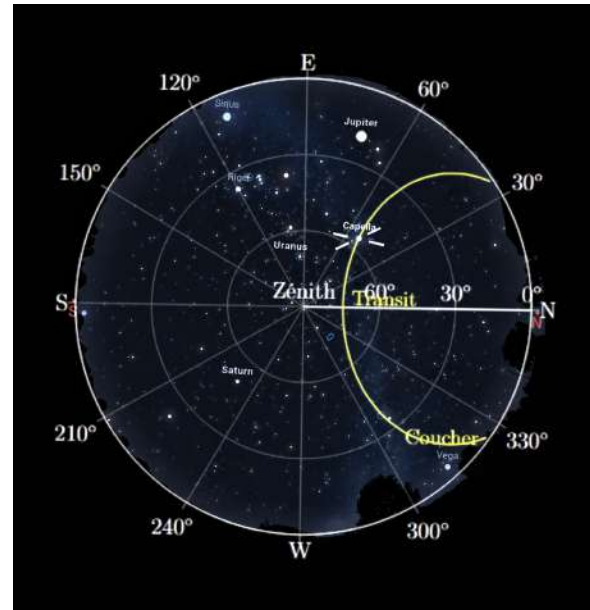


FIGURE 3.29 – La validité de l'algorithme sur Stellarium.

Phénomène de rotation de champ

À noter que sur une monture alt-azimutale, l'image rotée dans le plan focal au cours du suivi (phénomène dit de **field rotation**). Pour l'astrophotographie longue pose, un *dérotateur de champ* mécanique serait nécessaire — cette fonctionnalité dépasse le cadre du présent prototype mais constitue une perspective d'amélioration.

3.5.5 Le Mode Manuel (Joystick)

Définition et utilité

Le **mode manuel** permet à l'utilisateur de piloter directement les deux moteurs via le joystick analogique de la télécommande, en désactivant tout calcul trigonométrique. Ce mode est essentiel pour trois usages :

- (a) **Pointage initial (aiming the LOS)** : aligner manuellement la ligne de visée (Line of Sight) sur une étoile de référence avant d'entamer un suivi automatique ;
- (b) **Centrage de l'objet** : corriger l'éventuelle erreur résiduelle (jusqu'à 1°) du mode automatique due au backlash mécanique ;
- (c) **Calibrage et homing** : vérifier la procédure de fin de course et établir une nouvelle origine.

Principe de fonctionnement

En mode manuel, l'Arduino n°1 désactive les calculs sphériques et transmet directement, via la broche logique D9 (mise à HIGH), un signal d'activation à l'Arduino n°2. Ce dernier scrute alors continuellement les valeurs analogiques des axes X et Y du joystick (résolution 10 bits, soit 0 à 1023), centrées autour de la valeur de repos = 512. Un seuil (threshold) de 50 unités est appliqué autour de ce centre pour définir une zone morte (deadzone) qui prévient les dérives parasites dues au bruit analogique et aux légères variations mécaniques du joystick

$$v_{\text{moteur}} = \begin{cases} -v_{\text{max}} & \text{si } X < 512 - \tau \\ 0 & \text{si } |X - 512| \leq \tau \\ +v_{\text{max}} & \text{si } X > 512 + \tau \end{cases} \quad \text{avec } \tau = 50, v_{\text{max}} = 500 \text{ pas/s} \quad (3.28)$$

Extrait de code commenté

```

1 // Lecture analogique 10 bits (0      1023, repos = 512)
2 int joystickXValue = analogRead(joystickXPin);
3 int joystickYValue = analogRead(joystickYPin);
4
5 // Moteur 1 (Azimut) piloté par l'axe X
6 if (joystickXValue < (512 - threshold)) {
7     motor1.setSpeed(-maxSpeed); // rotation horaire
8 } else if (joystickXValue > (512 + threshold)) {
9     motor1.setSpeed(maxSpeed); // rotation antihoraire
10 } else {
11     motor1.setSpeed(0); // zone morte : arrêt
12 }
13
14 // Moteur 2 (Altitude) piloté par l'axe Y
15 if (joystickYValue < (512 - threshold)) {
16     motor2.setSpeed(-maxSpeed);
17 } else if (joystickYValue > (512 + threshold)) {
18     motor2.setSpeed(maxSpeed);
19 } else {
20     motor2.setSpeed(0);
21 }
22
23 // Exécution simultanée des deux moteurs en mode "continuous
    speed"
24 motor1.runSpeed();
25 motor2.runSpeed();

```

Listing 3.4 – Logique de pilotage joystick (Arduino n°2).

Caractérisation expérimentale

Le tableau 3.6 caractérise les performances du mode manuel :

TABLE 3.6 – Performances du mode manuel.

Paramètre	Valeur
Résolution joystick	10 bits (1024 niveaux)
Centre nominal	512 ± 5
Seuil (deadzone)	± 50 ($\approx \pm 5\%$)
Vitesse max	500 pas/s
Vitesse angulaire Az max	$\approx 0,79^\circ/\text{s}$
Vitesse angulaire Alt max	$\approx 3,22^\circ/\text{s}$
Temps de réaction	~ 50 ms

3.6 Système d'alimentation et dimensionnement

3.6.1 Bilan de puissance détaillé

Le dimensionnement de l'alimentation est une étape critique pour garantir le bon fonctionnement du système. Le bilan de puissance est dominé par les deux moteurs NEMA 17. Un moteur consomme théoriquement 1,7 A par phase, mais sous un pilotage en microstepping et en l'absence de charge nominale continue, le courant moyen consommé avoisine 0,4 à 0,6 A. L'Arduino Nano consomme ≈ 50 mA et le LCD I2C ≈ 20 mA.

TABLE 3.7 – Bilan détaillé des consommations électriques.

Élément	Tension	I moyen	I crête	P moyen
Moteur NEMA 17 Az	12 V	0,4 A	1,7 A	4,8 W
Moteur NEMA 17 Alt	12 V	0,4 A	1,7 A	4,8 W
Driver A4988 ($\times 2$)	5 V	20 mA	30 mA	0,2 W
Arduino Nano #1	5 V	50 mA	80 mA	0,25 W
Arduino Nano #2	5 V	50 mA	80 mA	0,25 W
LCD I2C	5 V	20 mA	40 mA	0,1 W
Total moyen				$\approx 10,4$ W

3.6.2 Solution principale : alimentation secteur 230 V / 12 V

Pour une utilisation en observatoire fixe ou dans tout lieu disposant d'une prise secteur, le montage est alimenté par un **adaptateur secteur 230 V $\rightarrow$ 12 V** à

découpage. Le choix du modèle repose sur le courant crête total du système.

Le courant total crête sur le rail 12 V est dominé par les deux moteurs :

$$I_{\text{crête, 12V}} = 2 \times 1,7 \text{ A} = 3,4 \text{ A} \quad (3.29)$$

En ajoutant la consommation de la logique ramenée sur le rail 12 V via le régulateur 5 V (rendement $\eta \approx 0,85$) :

$$I_{\text{logique, 12V}} = \frac{P_{\text{logique}}}{V \cdot \eta} = \frac{(0,2 + 0,25 + 0,25 + 0,1 + 0,15) \text{ W}}{12 \text{ V} \times 0,85} \approx 0,09 \text{ A} \quad (3.30)$$

Le courant total crête sur le rail 12 V est donc :

$$I_{\text{total, crête}} = 3,4 + 0,09 \approx 3,5 \text{ A} \quad (3.31)$$

En appliquant un coefficient de sécurité de 1,5 :

$$I_{\text{alim}} \geq 1,5 \times 3,5 = 5,25 \text{ A} \quad (3.32)$$

Un adaptateur secteur **12 V – 5 A** (60 W) est donc largement suffisant pour couvrir le fonctionnement nominal et les appels de courant transitoires, avec une puissance délivrée :

$$P_{\text{alim}} = 12 \text{ V} \times 5 \text{ A} = 60 \text{ W} \gg P_{\text{crête}} \approx 24 \text{ W} \quad (3.33)$$

Cette solution est simple, économique et ne présente aucune contrainte d'autonomie tant qu'une prise 230 V est disponible.

3.6.3 Amélioration : alimentation sur batterie Li-ion pour usage nomade

Lorsque le télescope est utilisé en extérieur, loin de toute prise secteur (site d'observation en montagne, désert, etc.), l'adaptateur secteur ne peut plus être employé. Une **batterie Li-ion** prend alors le relais, en délivrant directement la tension 12 V nécessaire.

Énergie requise pour une soirée d'observation

Pour une session d'observation de $T = 4$ heures, l'énergie consommée est :

$$E = P_{\text{moy}} \cdot T = 10,8 \text{ W} \times 4 \text{ h} = 43,2 \text{ Wh} \quad (3.34)$$

En appliquant un coefficient de sécurité de 1,5 (marge pour les pics de courant et le

vieillissement de la batterie) :

$$E_{\text{batterie}} \geq 1,5 \times 43,2 \approx 65 \text{ Wh} \quad (3.35)$$

Choix de la batterie

TABLE 3.8 – Comparaison des solutions de batterie pour usage nomade.

Type	Tension	Capacité	Énergie	Masse	Critère
Li-ion 3S	11,1 V	5000 mAh	55,5 Wh	350 g	< 65 Wh
Li-ion 4S	14,8 V	5000 mAh	74 Wh	450 g	> 65 Wh
Pb scellé	12 V	7 Ah	84 Wh	2200 g	Trop lourd

La **batterie Li-ion 4S 5000 mAh** (74 Wh) est retenue : elle est la seule à satisfaire simultanément le critère énergétique (74 Wh > 65 Wh) et la contrainte de masse pour un usage portable. L'autonomie effective estimée est :

$$T_{\text{autonomie}} = \frac{E_{\text{batterie}}}{P_{\text{moy}}} = \frac{74 \text{ Wh}}{10,8 \text{ W}} \approx 6,9 \text{ h} \quad (3.36)$$

Ce qui représente une marge confortable de +73% au-delà des 4 heures requises. Une carte BMS (Battery Management System) intégrée protège contre les sur-décharges, surcharges et courts-circuits.

3.6.4 Architecture de l'alimentation

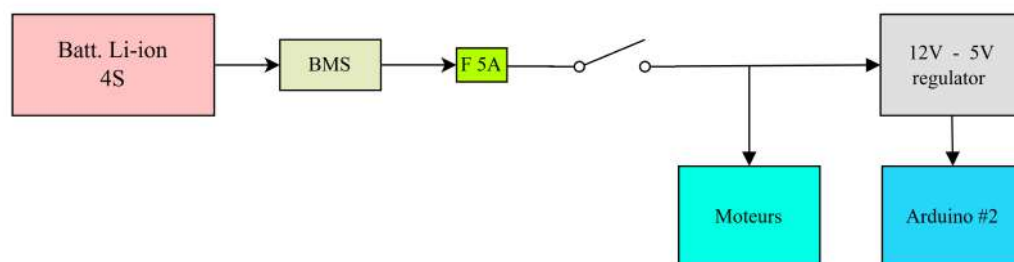


FIGURE 3.30 – Architecture simplifié du circuit d'alimentation : 12 V → drivers (direct) et → 5 V pour la logique (arduino).

3.7 La Touche Finale et Résultats

3.7.1 Le support laser vert (« Finder »)

Un support imprimé en 3D, précisément aligné avec l'axe optique du télescope, est prévu pour accueillir un laser de guidage. Ce dispositif est crucial pour :



FIGURE 3.31 – Laser de guidage

1. Effectuer un **alignement grossier** sur une étoile brillante de référence avant de basculer en mode hors-ligne ou en ligne ;
2. Vérifier visuellement la **précision de pointage** : si après une commande GoTo le laser tombe à $0,5^\circ$ de l'étoile cible, l'erreur résiduelle est immédiatement quantifiable ;
3. Servir d'outil pédagogique lors de soirées d'observation publiques pour montrer aux spectateurs où se situe l'objet pointé.

3.7.2 LOGO

Un logo a été conçu afin de renforcer l'identité visuelle et le professionnalisme du projet.



FIGURE 3.32 – LOGO

3.7.3 Tests sur le terrain et observations réalisées

Les tests sur le terrain ont été conduits depuis le site d'Agadir ($\phi = 30,4^\circ\text{N}$, $\lambda = -9,6^\circ$, altitude 60 m) au cours de plusieurs soirées d'observation entre l'automne et l'hiver 2024-2025. Les cibles principales observées avec succès sont :

- **La Lune** : observation détaillée des cratères dans la région de l'Oceanus Procellarum (l'Océan des Tempêtes), avec une bonne résolution
- **La comète C/2023 A3 (Tsuchinshan-ATLAS)** : acquisition manuelle réussie en octobre 2024 lors de son passage proche de la Terre ;
- **Étoiles brillantes du catalogue** : Sirius, Vega, Altaïr, Aldebaran ;
- **Amas d'étoiles** : les Pléiades (M45), observables comme un groupe d'étoiles brillantes facilement identifiable ;
- **Planètes** : Jupiter et Saturne, identifiables parmi les objets les plus lumineux du ciel nocturne. Jupiter apparaît comme un petit disque lumineux avec éventuellement ses quatre principaux satellites galiléens dans de bonnes conditions, tandis que Saturne présente une forme légèrement allongée.

3.7.4 Précision de pointage mesurée

La précision de pointage a été quantifiée en comparant la position visée (selon les coordonnées Stellarium) à la position effectivement atteinte par le télescope (validée à l'oculaire et au laser). Le graphique suivant présente l'erreur angulaire mesurée sur 10 cibles :

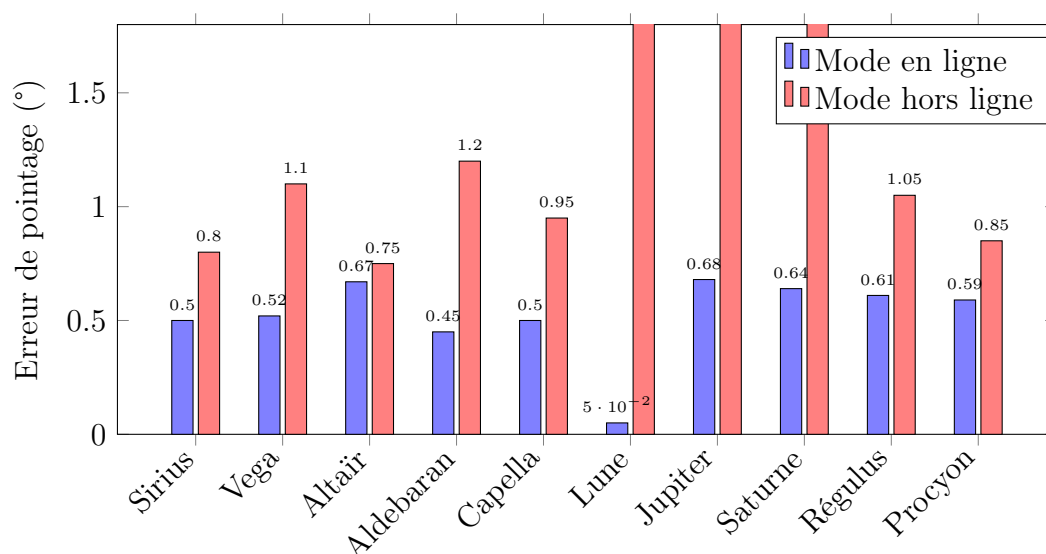


FIGURE 3.33 – Erreur de pointage en mode en ligne (bleu) et hors-ligne (rouge) pour 10 cibles célestes depuis Agadir.

Remarque

ces erreurs ont été obtenues par une combinaison de simulations et de tests réels effectués en journée, et non de nuit

Pour une comparaison équitable entre les deux modes, les objets du système solaire la Lune, Jupiter et Saturne ont été exclus de l'analyse, car leurs positions évoluent trop rapidement pour être calculées avec précision en mode horsligne à partir des coordonnées, ce qui engendre des erreurs anormalement élevées non représentatives du comportement général du système. En ne retenant que les cibles stellaires fixes, l'erreur moyenne de pointage en mode en ligne s'établit à **0,55°**, tandis qu'elle atteint **0,96°** en mode hors-ligne. avec un rapport d'environ 1,74, reste néanmoins acceptable compte tenu du grossissement modeste de $12\times$ du télescope artisanal, pour lequel une telle précision de pointage demeure tout à fait suffisante.

généralement ces erreurs sont liées principalement à :

- Le **backlash mécanique** (jeu d'engrenage) des pignons imprimés en 3D, particulièrement marqué lors d'un changement de sens de rotation ;
- L'**erreur d'orthogonalité** entre les deux axes (mesure imparfaite à la construction) ;

3.7.5 Photos du prototype et des observations

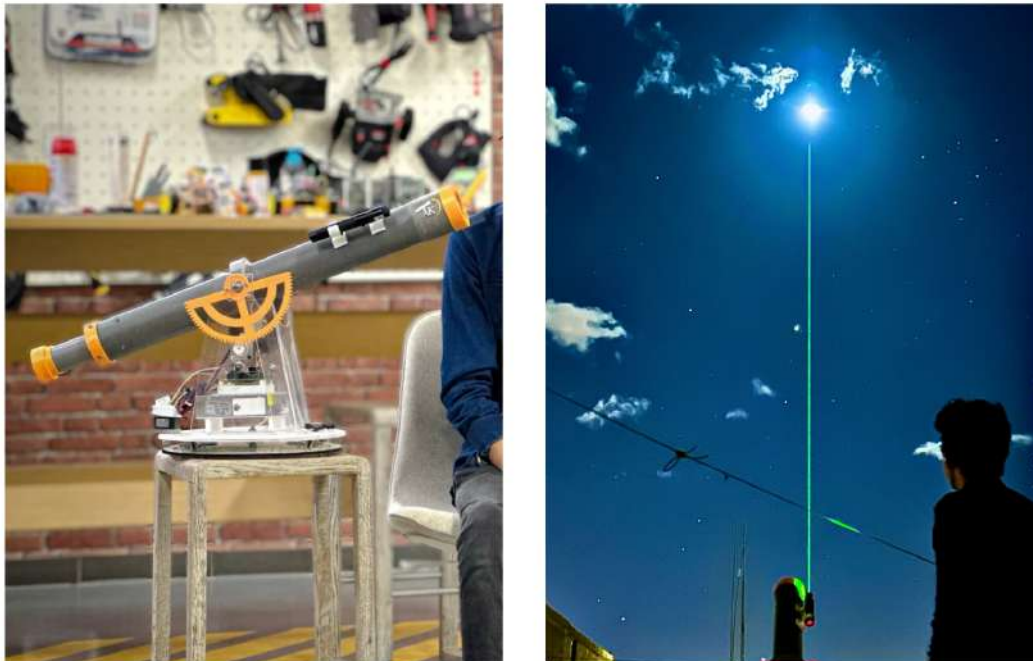


FIGURE 3.34 – Télescope assemblé au FabLab et en cours de test nocturne à Agadir.



FIGURE 3.35 – À gauche : photographie de la Lune réalisée à travers l’oculaire avec un smartphone. Au centre : cliché de la planète Saturne. À droite : comète C/2023 A3 capturée pendant son passage en octobre 2024.

Remarque

Ces photos ont été prises avec la caméra d’un téléphone, mais à l’observation à l’œil nu, la vue apparaissait beaucoup plus détaillée.

3.7.6 Perspectives d'amélioration

Plusieurs axes d'amélioration ont été identifiés pour de futures itérations :

1. **Télescope secondaire 80×** : intégration d'un instrument secondaire à plus fort grossissement, monté en parallèle, pour observation détaillée après pointage par l'instrument principal ;
2. **Motorisation du focuser** (mise au point) : remplacer la mise au point manuelle par un troisième moteur stepper piloté à distance ;
3. **Encodeurs absolus** sur les deux axes pour fermer la boucle d'asservissement et éliminer la sensibilité au backlash ;
4. **Module GPS** pour acquérir automatiquement la latitude, longitude et l'heure UT, supprimant la phase de saisie manuelle ;
5. **Caméra CCD/CMOS** avec interface USB pour pratiquer l'astrophotographie planétaire et l'autoguidage par étoile-guide ;
6. **Application mobile** (Bluetooth ou Wi-Fi) pour piloter le télescope depuis un smartphone, avec connexion à Stellarium Mobile.

Conclusion Générale

Le projet "Étude et réalisation d'un Télescope Dobsonien Autoguidé" a constitué une excellente opportunité de mettre en pratique et d'approfondir les compétences acquises durant ma formation à la FST de Tanger. Il a exigé de la rigueur à travers un large spectre de disciplines de l'ingénieur : conception mécanique et modélisation sur Onshape, théorie optique, électronique de puissance avec isolation des signaux, et calcul algorithmique complexe basé sur la trigonométrie sphérique.

Les objectifs initiaux ont été atteints : le télescope est fonctionnel, son boîtier de télécommande permet de naviguer dans le catalogue astronomique, et la communication Python-Stellarium offre une connectivité étendue. Le support précieux du FabLab d'Orange Digital Center et les composants/conseils de Moussasoft ont indéniablement catalysé le processus de réalisation.

Parmi les limites actuelles du prototype figurent les tolérances mécaniques des pignons imprimés en 3D qui introduisent un léger "backlash" (jeu d'engrenage), réduisant la précision absolue du pointage automatique. En termes de perspectives, il serait judicieux de remplacer la courroie crantée par un engrenage usiné en aluminium ou d'implémenter un algorithme de correction de backlash dans le code Arduino. De plus, l'intégration d'un encodeur rotatif permettrait une boucle fermée (closed-loop) pour l'asservissement, rendant le système GoTo totalement autonome face aux erreurs de positionnement physique.

En conclusion, ce projet fut une formidable aventure scientifique et humaine, me préparant efficacement aux exigences complexes des systèmes mécatroniques et de l'ingénierie moderne.

Bibliographie

- [1] W.M. Smart, *Text-Book on Spherical Astronomy*, Cambridge University Press, 6th Edition, 1977.
- [2] Jean Meeus, *Astronomical Algorithms*, Willmann-Bell, 2nd Edition, 1998.
- [3] Documentation officielle Arduino, *Arduino Nano & Libraries*, <https://www.arduino.cc/en/Reference/HomePage>.
- [4] Mike McCauley, *AccelStepper Library for Arduino*, <https://www.airspayce.com/mikem/arduino/AccelStepper/>.
- [5] Stellarium Web Online Planetarium, *Web Service API*, <https://stellarium-web.org/>.
- [6] Onshape CAD, PTC, *Cloud-based 3D CAD*, <https://www.onshape.com>.
- [7] Moussasoft E-commerce, *Composants électroniques, Arduino et Robotique*, <https://www.moussasoft.com/>.
- [8] Orange Digital Center Maroc, *FabLab et Écosystème digital*, <https://www.orangedigitalcenters.com/country/ma/home>.
- [9] AK-HAIL Oussama, *Autoguided Telescope Instructables*, ODC Agadir, 2024.
- [10] Selenium Webdriver for Python, *Browser Automation*, <https://selenium-python.readthedocs.io/>.
- [11] NASA JPL Horizons, *Ephemeris System*, <https://ssd.jpl.nasa.gov/horizons/>.
- [12] Rutten, H. et van Venrooij, M., *Telescope Optics : Evaluation and Design*, Willmann-Bell, 1988.
- [13] H. Karttunen, P. Kröger, H. Oja, M. Poutanen, K.J. Donner, *Fundamental Astronomy*, Springer-Verlag, 6th Edition, 2017.
- [14] P. Duffett-Smith, *Practical Astronomy with your Calculator*, Cambridge University Press, 3rd Edition, 1988.
- [15] Allegro MicroSystems, *A4988 Stepper Motor Driver Datasheet*, 2014.

Chapitre A

Code complet : Arduino n°1 (Télécommande)

Ce premier microcontrôleur gère le LCD, le joystick, les boutons, les calculs astronomiques (mode hors-ligne) et la communication avec l'Arduino n°2.

```
1  /* Arduino n 1 : remote, calculs et IHM */
2  #include
3  #include
4  #include
5
6  LCD_I2C lcd(0x27, 16, 2);
7
8  const int stepPin1 = 2, dirPin1 = 3;
9  const int stepPin2 = 5, dirPin2 = 4;
10 AccelStepper motor1(AccelStepper::DRIVER, stepPin1, dirPin1);
11 AccelStepper motor2(AccelStepper::DRIVER, stepPin2, dirPin2);
12
13 int joystickXPin = A1, joystickYPin = A0;
14 int joystickButtonPin = 12;
15 int leftButtonPin = 11, rightButtonPin = 10;
16 int pindyaljoy = 9, pindyalonline = 8;
17
18 int modeState = 0, settingState = 0, scopeState = 0;
19 int year = 2020, month = 1, day = 1, hour = 0, minute = 0, second =
    0;
20 float prealt = 0, preaz = 0;
21
22 struct SkyObject { String name; float RA; float Dec; };
23 SkyObject skyObjects[20] = {
24     {"Sirius", 6.7525, -16.7161}, {"Canopus", 6.3992, -52.6957},
25     {"Arcturus", 14.2610, 19.1825}, {"Vega", 18.6156, 38.7837},
26     {"Capella", 5.2782, 45.9979}, {"Rigel", 5.2423, -8.2016},
27     {"Procyon", 7.6550, 5.2250}, {"Achernar", 1.6286, -57.2368},
```



```
28   {"Betelgeuse", 5.9195, 7.4071}, {"Hadar", 14.0637, -60.3729},
29   {"Altair", 19.8463, 8.8683}, {"Aldebaran", 4.5987, 16.5092},
30   {"Antares", 16.4901, -26.4320}, {"Spica", 13.4199, -11.1613},
31   {"Pollux", 7.7553, 28.0262}, {"Fomalhaut", 22.9608, -29.6222},
32   {"Deneb", 20.6905, 45.2803}, {"Regulus", 10.1395, 11.9672},
33   {"Castor", 7.5766, 31.8884}, {"Alnair", 22.1372, -46.9601}
34 };
35 int selectedObject = 0;
36
37 void setup() {
38   digitalWrite(pindyaljoy, LOW);
39   lcd.begin(); lcd.display(); lcd.backlight();
40   pinMode(joystickButtonPin, INPUT_PULLUP);
41   pinMode(leftButtonPin, INPUT);
42   pinMode(rightButtonPin, INPUT_PULLUP);
43   pinMode(pindyaljoy, OUTPUT);
44   pinMode(pindyalonline, OUTPUT);
45   lcd.setCursor(4,0); lcd.print("AK space"); delay(600);
46   lcd.setCursor(4,1); lcd.print("AK-scope"); delay(1000); lcd.clear
   ();
47   settingState = 0; scopeState = 1;
48   Serial.begin(9600);
49 }
50
51 void loop() {
52   if (digitalRead(leftButtonPin) == HIGH) {
53     delay(200); modeState = (modeState == 0) ? 1 : 0; lcd.clear();
54   }
55   if (modeState == 0) { scope_mode(); }
56   else if (modeState == 1) {
57     lcd.setCursor(4,0); lcd.print("settings");
58     delay(600); handleSettingMode();
59   }
60 }
61
62 void handleSettingMode() {
63   switch (settingState) {
64     case 0: while (!joystickButtonPressed()) {
65       lcd.setCursor(0,0); lcd.print("Set Year: "); lcd.print(year)
66       ;
67       if (joystickUp() && year < 2030) year++;
68       if (joystickDown() && year > 2020) year--;
69       if (digitalRead(leftButtonPin) == HIGH) { modeState = 1;
70         break; }
71     } settingState++; lcd.clear(); delay(200); break;
72     case 1: while (!joystickButtonPressed()) {
```

```

71     lcd.setCursor(0,0); lcd.print("Set Month: "); lcd.print(
month);
72     if (joystickUp() && month < 12) month++;
73     if (joystickDown() && month > 1) month--;
74     if (digitalRead(leftButtonPin) == HIGH) { modeState = 1;
break; }
75     } settingState++; lcd.clear(); delay(200); break;
76     case 2: while (!joystickButtonPressed()) {
77         lcd.setCursor(0,0); lcd.print("Set Day: "); lcd.print(day);
78         if (joystickUp() && day < 31) day++;
79         if (joystickDown() && day > 1) day--;
80         if (digitalRead(leftButtonPin) == HIGH) { modeState = 1;
break; }
81     } settingState++; lcd.clear(); delay(200); break;
82     case 3: while (!joystickButtonPressed()) {
83         lcd.setCursor(0,0); lcd.print("Set Hour: "); lcd.print(hour)
;
84         if (joystickUp() && hour < 23) hour++;
85         if (joystickDown() && hour > 0) hour--;
86         if (digitalRead(leftButtonPin) == HIGH) { modeState = 1;
break; }
87     } settingState++; lcd.clear(); delay(200); break;
88     case 4: while (!joystickButtonPressed()) {
89         lcd.setCursor(0,0); lcd.print("Set Minute: "); lcd.print(
minute);
90         if (joystickUp() && minute < 59) minute++;
91         if (joystickDown() && minute > 0) minute--;
92         if (digitalRead(leftButtonPin) == HIGH) { modeState = 1;
break; }
93     } settingState++; lcd.clear(); delay(200); break;
94     case 5: while (!joystickButtonPressed()) {
95         lcd.setCursor(0,0); lcd.print("Set Second: "); lcd.print(
second);
96         if (joystickUp() && second < 59) second++;
97         if (joystickDown() && second > 0) second--;
98         if (digitalRead(leftButtonPin) == HIGH) { modeState = 1;
break; }
99     } settingState = 0; modeState = 0; lcd.clear(); break;
100 }
101 }
102
103 bool joystickUp() { return (analogRead(joystickYPin) > 400) ? (
delay(200), true) : false; }
104 bool joystickDown() { return (analogRead(joystickYPin) < 600) ? (
delay(200), true) : false; }
105 bool joystickButtonPressed() {

```

```

106     return (digitalRead(joystickButtonPin) == LOW) ? (delay(200), true
107         ) : false;
108 }
109 void handleOfflineMode() {
110     lcd.setCursor(3,0); lcd.print(skyObjects[selectedObject].name);
111     if (joystickUp() && selectedObject < 19) { selectedObject++; lcd.
112         clear(); }
113     if (joystickDown() && selectedObject > 0) { selectedObject--; lcd.
114         clear(); }
115     if (joystickButtonPressed()) calculation();
116 }
117 float degreesToRadians(float d) { return d * PI / 180.0; }
118 float radiansToDegrees(float r) { return r * 180.0 / PI; }
119 float calculateGST(int year, int month, int day, int hour, int
120     minute, int second) {
121     float UT = hour - 7 + minute - 15 / 60.0 + second / 3600.0;
122     int yTemp = year + (month + 9) / 12;
123     float d = 367 * year - (7 * yTemp) / 4 + (275 * month) / 9 + day -
124         730530;
125     float GST = 6.697374558 + 0.06570982441908 * d + 1.00273790935 *
126         UT;
127     return fmod(GST, 24.0);
128 }
129 float calculateLST(float gst, float longitude) {
130     float LST = gst + longitude / 15.0;
131     return fmod(LST, 24.0);
132 }
133 void calculateAltAz(float ra, float dec, float lat, float lst, float
134     &alt, float &az) {
135     float HA = fmod((lst * 15.0 - ra * 15.0), 360.0);
136     if (HA < 0) HA += 360.0;
137     HA = degreesToRadians(HA);
138     dec = degreesToRadians(dec);
139     lat = degreesToRadians(lat);
140     float sinAlt = sin(dec)*sin(lat) + cos(dec)*cos(lat)*cos(HA);
141     alt = asin(sinAlt);
142     float cosAz = (sin(dec) - sin(alt)*sin(lat)) / (cos(alt)*cos(lat))
143         ;
144     cosAz = constrain(cosAz, -1, 1);
145     az = acos(cosAz);
146     alt = radiansToDegrees(alt); az = radiansToDegrees(az);
147     if (sin(HA) > 0) az = 360.0 - az;

```

```
145 }
146
147 float calculation() {
148     float latitude = 30.25, longitude = -9.36;
149     float RA = skyObjects[selectedObject].RA;
150     float Dec = skyObjects[selectedObject].Dec;
151     lcd.clear(); lcd.setCursor(0,0); lcd.print("Calculating...");
152     float gst = calculateGST(year, month, day, hour, minute, second);
153     float lst = calculateLST(gst, longitude);
154     float alt, az; calculateAltAz(RA, Dec, latitude, lst, alt, az);
155     char buffer[10] = "";
156     sprintf(buffer, "%d %d ", (int)az, (int)alt);
157     Serial.println(buffer);
158     lcd.clear(); lcd.print("az: "); lcd.print(az);
159     lcd.setCursor(0,1); lcd.print("alt: "); lcd.print(alt);
160     delay(2000); lcd.clear();
161     return az, alt;
162 }
163
164 void scope_mode() {
165     switch (scopeState) {
166         case 1: // Offline
167             while (digitalRead(rightButtonPin) == 1) {
168                 digitalWrite(pindyaljoy, LOW); digitalWrite(pindyalonline,
169                     LOW);
170                 handleOfflineMode();
171                 if (digitalRead(leftButtonPin) == HIGH) { modeState = 0;
172                     break; }
173                 if (digitalRead(rightButtonPin) == 0) { scopeState = 2;
174                     delay(200); break; }
175             } break;
176         case 2: // Online
177             lcd.setCursor(2,0); lcd.print("online mode: ");
178             lcd.setCursor(0,1); lcd.print("use the software");
179             while (digitalRead(rightButtonPin) == 1) {
180                 digitalWrite(pindyaljoy, LOW); digitalWrite(pindyalonline,
181                     HIGH);
182                 if (digitalRead(leftButtonPin) == HIGH) { modeState = 0;
183                     break; }
184                 if (digitalRead(rightButtonPin) == 0) { scopeState = 3;
185                     delay(200); break; }
186             } break;
187         case 3: // Joystick
188             lcd.setCursor(1,0); lcd.print("joystick mode");
189             while (digitalRead(rightButtonPin) == 1) {
190                 digitalWrite(pindyaljoy, HIGH); digitalWrite(pindyalonline,
191                     LOW);
```

```
185     if (digitalRead(leftButtonPin) == HIGH) { modeState = 0;
    break; }
186     if (digitalRead(rightButtonPin) == 0) { scopeState = 1;
    digitalWrite(pindyajoy, LOW); delay(200); break; }
187     } break;
188 }
189 }
```

Listing A.1 – Firmware complet de l'Arduino n°1 (Télécommande, calculs et IHM).

Chapitre B

Code complet : Arduino n°2 (Pilote des moteurs)

Ce second microcontrôleur reçoit les consignes (par série) ou les commandes joystick, et pilote directement les deux moteurs pas-à-pas via les drivers A4988.

```
1  /* Arduino n 2 : commande moteurs */
2  #include
3
4  String inputString = "";
5  int azimuth = 0, altitude = 0;
6
7  const int stepPin1 = 2, dirPin1 = 3;
8  int prexx = 0, prezz = 0;
9  const int stepPin2 = 5, dirPin2 = 4;
10 int prealt = 0, preaz = 0;
11
12 const int joystickXPin = A0, joystickYPin = A1;
13 int pindyaljoy = 10, pindyalonline = 11;
14 const int threshold = 50;
15
16 AccelStepper motor1(AccelStepper::DRIVER, stepPin1, dirPin1);
17 AccelStepper motor2(AccelStepper::DRIVER, stepPin2, dirPin2);
18 const int maxSpeed = 500, accel = 500;
19
20 void setup() {
21   pinMode(6, INPUT);
22   pinMode(pindyaljoy, INPUT);
23   pinMode(pindyalonline, INPUT);
24   motor1.setMaxSpeed(maxSpeed); motor1.setAcceleration(accel);
25   motor2.setMaxSpeed(maxSpeed); motor2.setAcceleration(accel);
26   Serial.begin(9600);
27   long steps007 = map(40 * 2, 0, 360, 0, 200 * 32);
28   motor1.moveTo(-steps007 * 2.2);
```

```

29   if (digitalRead(6) == HIGH) {
30       while (motor1.distanceToGo() != 0) motor1.run();
31   }
32 }
33
34 void loop() {
35     if (digitalRead(pindyaljoy) == LOW) {
36         if (Serial.available() > 0) {
37             inputString = Serial.readStringUntil('\n');
38             Serial.println(inputString);
39             int i1 = inputString.indexOf(' ');
40             int i2 = inputString.lastIndexOf(' ');
41             if (i1 != -1 && i2 != -1 && i1 != i2) {
42                 azimuth = inputString.substring(0, i1).toInt();
43                 altitude = inputString.substring(i1+1, i2).toInt();
44                 stepstep(altitude, azimuth);
45                 altitude = 0; azimuth = 0;
46             }
47         }
48     } else if (digitalRead(pindyalonline) == HIGH && digitalRead(
pindyaljoy) == LOW) {
49         if (Serial.available()) {
50             inputString = Serial.readString();
51             // (parsing ASCII complet : deg min ' sec ")
52             int f1 = inputString.indexOf(' ');
53             int f2 = inputString.indexOf('\ ');
54             int f3 = inputString.indexOf('\ " ');
55             String altDeg = inputString.substring(0, f1);
56             String altMin = inputString.substring(f1+1, f2);
57             String altSec = inputString.substring(f2+1, f3);
58             int xx = altDeg.toInt() - prexx;
59             prexx = altDeg.toInt();
60             int zz = inputString.substring(f3+1, inputString.indexOf(' ',
f1+1)).toInt();
61             stepstep(xx, zz);
62         }
63     } else if (digitalRead(pindyaljoy) == HIGH && digitalRead(
pindyalonline) == LOW) {
64         int xv = analogRead(joystickXPin);
65         int yv = analogRead(joystickYPin);
66         if (xv < (512 - threshold)) motor1.setSpeed(-maxSpeed);
67         else if (xv > (512 + threshold)) motor1.setSpeed(maxSpeed);
68         else motor1.setSpeed(0);
69         if (yv < (512 - threshold)) motor2.setSpeed(-maxSpeed);
70         else if (yv > (512 + threshold)) motor2.setSpeed(maxSpeed);
71         else motor2.setSpeed(0);
72         motor1.runSpeed(); motor2.runSpeed();

```

```
73   }
74 }
75
76 void steptep(int a, int b) {
77   a = a - prealt;
78   b = b - preaz;
79   long stepsAz = map(b * 2, 0, 360, 0, 200 * 32);
80   long stepsAlt = map(a * 2, 0, 360, 0, 200 * 32);
81   motor2.moveTo(motor2.currentPosition() + stepsAz * 8.18);
82   motor1.moveTo(motor1.currentPosition() + stepsAlt * 2.2);
83   while (motor1.distanceToGo() != 0 || motor2.distanceToGo() != 0) {
84     if (digitalRead(6) == LOW) { motor1.run(); }
85     else if (digitalRead(6) == HIGH) {
86       motor1.stop(); b = 40; preaz = 40; break;
87     }
88     motor2.run();
89   }
90   prealt = a + prealt; preaz = b + preaz;
91 }
```

Listing B.1 – Firmware complet de l'Arduino n°2 (commande moteurs).

Chapitre C

Code complet : Application Python (Stellarium Scraper)

L'application Python tourne sur le PC de l'utilisateur. Elle fournit une interface graphique (Tkinter), scrape Stellarium Web via Selenium, et transmet les coordonnées par UART à l'Arduino n°2.

```
1 import tkinter as tk
2 from tkinter import ttk, scrolledtext
3 from selenium import webdriver
4 from selenium.webdriver.chrome.service import Service
5 from selenium.webdriver.common.by import By
6 from selenium.webdriver.support.ui import WebDriverWait
7 from selenium.webdriver.support import expected_conditions as EC
8 from selenium.common.exceptions import TimeoutException
9 import threading, sys, glob, time
10 from PIL import Image, ImageTk
11 import serial
12
13 class StellariumApp:
14     def __init__(self, master):
15         self.master = master
16         master.title("Stellarium Web Scraper")
17         master.geometry("600x400")
18         master.configure(bg='navy')
19         self.sky_objects = [
20             "Sun", "Moon", "Mercury", "Venus", "Mars", "Jupiter", "
Saturn",
21             "Uranus", "Neptune", "Pluto", "Sirius", "Betelgeuse", "
Rigel",
22             "Vega", "Antares", "Aldebaran", "Pollux", "Spica", "
Arcturus",
23             "Capella", "Procyon", "Achernar", "Deneb", "Regulus", "
Altair",
```

```
24         "Pleiades", "Orion Nebula", "Andromeda Galaxy", "Crab
Nebula",
25         "Whirlpool Galaxy", "Triangulum Galaxy", "Sombrero
Galaxy",
26         "Centaurus A", "Carina Nebula", "Eagle Nebula", "Omega
Centauri",
27         "47 Tucanae", "Horsehead Nebula", "Crab Pulsar", "
Beehive Cluster",
28         "Dumbbell Nebula", "Ring Nebula", "Pinwheel Galaxy",
29         "Bode's Galaxy", "Cigar Galaxy", "Tarantula Nebula",
30         "Helix Nebula", "Lagoon Nebula", "Trifid Nebula",
31         "Pillars of Creation"
32     ]
33     self.set_background_image()
34     self.create_widgets()
35     self.animate_background()
36
37     def serial_ports(self):
38         global resultport
39         if sys.platform.startswith('win'):
40             ports = ['COM%s' % (i+1) for i in range(256)]
41         else:
42             raise EnvironmentError('Unsupported platform')
43         resultport = []
44         for port in ports:
45             try:
46                 s = serial.Serial(port); s.close()
47                 resultport.append(port)
48             except (OSError, serial.SerialException):
49                 pass
50         if len(resultport) == 0:
51             resultport.append("No device is connected")
52         return resultport
53
54     def set_background_image(self):
55         bg = Image.open("milky-way.jpeg")
56         bg = bg.resize((1200,800), Image.Resampling.LANCZOS)
57         self.bg_photo = ImageTk.PhotoImage(bg)
58         self.canvas = tk.Canvas(self.master, width=800, height=600)
59         self.canvas.pack(fill="both", expand=True)
60         self.bg_image_id = self.canvas.create_image(0, 0, image=self
.bg_photo, anchor="nw")
61
62     def create_widgets(self):
63         frame = tk.Frame(self.master, bg='#151f36')
64         frame.place(relx=0.5, rely=0.5, anchor='center')
65         ttk.Label(frame, text="Select a sky object:",
```

```
66         background='#151f36', foreground='white')).pack(
        pady=10)
67     ports = self.serial_ports()
68     self.object_var2 = tk.StringVar()
69     self.object_combo = ttk.Combobox(frame, textvariable=self.
    object_var2, values=ports)
70     self.object_combo.pack(pady=6); self.object_combo.set("
    select the port")
71     self.object_var = tk.StringVar()
72     self.object_combo = ttk.Combobox(frame, textvariable=self.
    object_var, values=self.sky_objects)
73     self.object_combo.pack(pady=5); self.object_combo.set(self.
    sky_objects[0])
74     ttk.Button(frame, text="Search", command=self.start_search).
    pack(pady=10)
75     self.result_text = scrolledtext.ScrolledText(frame, width
    =50, height=10, bg='#2e437c', fg='white')
76     self.result_text.pack(pady=10)
77
78     def start_search(self):
79         self.result_text.delete('1.0', tk.END)
80         self.result_text.insert(tk.END, "Searching... Please wait.\n
    ")
81         threading.Thread(target=self.search_object, daemon=True).
    start()
82
83     def search_object(self):
84         chosenport = self.object_var2.get()
85         arduino = serial.Serial(port=resultport[0], baudrate=115200,
    timeout=1)
86         name1 = self.object_var.get()
87         service = Service(executable_path='chromedriver.exe')
88         options = webdriver.ChromeOptions()
89         options.add_argument("--headless=new")
90         driver = webdriver.Chrome(service=service, options=options)
91         try:
92             driver.get('https://stellarium-web.org/')
93             WebDriverWait(driver, 20).until(
94                 EC.presence_of_element_located((By.CSS_SELECTOR, '
    body'))))
95             try:
96                 close_button = WebDriverWait(driver, 0.5).until(
97                     EC.element_to_be_clickable((By.CSS_SELECTOR, '
    button[title="Close"]'))))
98                 close_button.click()
99             except TimeoutException: pass
100             search_box = WebDriverWait(driver, 10).until(
```

```

101         EC.presence_of_element_located((By.ID, 'input-33'))
102         search_box.send_keys(name1); search_box.send_keys(u'\
ue007')
103         object_element = WebDriverWait(driver, 20).until(
104             EC.element_to_be_clickable((By.XPATH, f"//div[
contains(text(), '{name1}')]"))
105         object_element.click()
106         WebDriverWait(driver, 20).until(
107             EC.presence_of_element_located((By.CSS_SELECTOR, '.
row.no-gutters'))
108         radec_vals = driver.find_elements(By.CLASS_NAME, "
radecVal")
109         az = (radec_vals[9].text, radec_vals[10].text,
radec_vals[11].text)
110         alt = (radec_vals[6].text, radec_vals[7].text,
radec_vals[8].text)
111         result = f"{az[0]} {az[1]} {az[2]}\n{alt[0]} {alt[1]} {
alt[2]}\n"
112         arduino.write(bytes(result, 'utf-8')); time.sleep(0.05)
113         self.master.after(0, self.update_result, result)
114     except TimeoutException as e:
115         self.master.after(0, self.update_result, f"Error: {e}")
116     finally:
117         driver.quit()
118
119     def update_result(self, result):
120         self.result_text.delete('1.0', tk.END)
121         self.result_text.insert(tk.END, result)
122
123     def animate_background(self):
124         x, y = self.canvas.coords(self.bg_image_id)
125         new_x = x - 0.5
126         if new_x <= -800: new_x = 0
127         self.canvas.coords(self.bg_image_id, new_x, y)
128         self.master.after(50, self.animate_background)
129
130 if __name__ == "__main__":
131     root = tk.Tk()
132     app = StellariumApp(root)
133     root.mainloop()

```

Listing C.1 – Application Python (Tkinter + Selenium + PySerial).

Chapitre D

Schémas de câblage détaillés

D.1 Brochage Arduino n°1

TABLE D.1 – Brochage complet de l'Arduino n°1 (Télécommande).

Broche	Direction	Connexion
A0	IN analogique	Joystick Y
A1	IN analogique	Joystick X
A4	I2C SDA	LCD I2C
A5	I2C SCL	LCD I2C
D2	OUT	STEP1 (Az) (vers Arduino 2)
D3	OUT	DIR1 (Az)
D4	OUT	DIR2 (Alt)
D5	OUT	STEP2 (Alt)
D8	OUT	ONLINE_MODE (vers Ard2)
D9	OUT	JOY_MODE (vers Ard2)
D10	IN pull-up	Bouton « Scope »
D11	IN	Bouton « Mode »
D12	IN pull-up	Joystick switch
TX/RX	UART	Vers Arduino n°2
VIN	POWER	7-12 V

D.2 Brochage Arduino n°2

TABLE D.2 – Brochage complet de l’Arduino n°2 (Pilote moteurs).

Broche	Direction	Connexion
A0	IN analogique	Joystick X (recopié)
A1	IN analogique	Joystick Y (recopié)
D2	OUT	STEP du driver 1 (Az)
D3	OUT	DIR du driver 1 (Az)
D4	OUT	DIR du driver 2 (Alt)
D5	OUT	STEP du driver 2 (Alt)
D6	IN pull-up	Fin de course (homing)
D10	IN	JOY_MODE (depuis Ard1)
D11	IN	ONLINE_MODE (depuis Ard1)
TX/RX	UART	USB / Vers Arduino n°1

D.3 Câblage A4988

Chaque driver A4988 est câblé comme suit :

- VMOT et GND_{MOT} reliés au bus 12 V (avec condensateur de découplage 100 μ F) ;
- VDD et GND_{LOG} reliés au bus 5 V ;
- STEP et DIR reliés aux broches D2/D3 (driver 1) ou D5/D4 (driver 2) ;
- MS1, MS2, MS3 mises à HIGH pour configuration microstepping 1/32 ;
- ENABLE mis à LOW (drivers actifs) ;
- RESET et SLEEP reliés ensemble à HIGH ;
- Bobines moteur : 1A, 1B, 2A, 2B aux quatre fils du NEMA 17 ;
- Potentiomètre V_{ref} ajusté à 0,64 V pour $I_{max} = 0,8$ A.

D.4 Connecteur télécommande $\leftrightarrow$ télescope

Le câble blindé entre la télécommande et le boîtier électronique du télescope comporte 8 conducteurs :

- (a) +12 V (alimentation depuis batterie)
- (b) GND
- (c) +5 V (depuis UBEC interne du télescope)
- (d) TX (UART)

- (e) RX (UART)
- (f) JOY_MODE (D9 ↔ D10)
- (g) ONLINE_MODE (D8 ↔ D11)
- (h) Reserve / Bouton URGENCE